💻 Origini dell'informatica

Le Origini dell'Informatica

Contesto Storico Matematico (Fine '800)

- Periodo di profonda riflessione sui fondamenti matematici
- **David Hilbert** propone 23 problemi matematici fondamentali per il XX secolo

Problematiche Fondamentali

- **Domanda Centrale**: La matematica è un sistema formale completo?
- **Problema della Decisione**: Esistenza di un procedimento "meccanico" per dimostrare conseguenze logiche

Contributi Teorici Rivoluzionari

Gödel: Teorema di Incompletezza (1931)

- In sistemi formali sufficientemente complessi esistono affermazioni:
  - **Non dimostrabili** come vere
  - Non dimostrabili come false
  - Basate sugli assiomi del sistema stesso

Church-Turing-Kleene: Formalizzazione di "metodo meccanico"

- **Macchina di Turing** (1936, TM) come modello computazionale fondamentale
- **Tesi di Church-Turing**: Tutto ciò che è computabile può essere eseguito da una macchina di Turing universale
- **Computer attuali:** Hanno capacità computazionale equivalente a una TM, ma se un problema non è risolvibile con una TM non lo è nemmeno con un supercomputer

Algoritmi: Concetti Fondamentali

Definizione di Algoritmo

- **Descrizione di un metodo di soluzione per un problema che:**
  - Sia **eseguibile** (sequenza di passi definiti)
  - **Privo di ambiguità**
  - Arrivi a conclusione in **numero finito di passi**

Componenti Algoritmiche

- **Sequenze**: Modalità di organizzazione di **informazioni atomiche** → salvate in **data container**
- **Tipologie di Numeri**:
  - Naturali
  - Interi
  - Razionali
  - Reali

Operatori e Decomposizione

- **Scomposizione di problemi complessi in operazioni semplici**
- Gestione di **criticità negli operatori** (es. divisione)
- Utilizzo di **iterazione** per risolvere problemi complessi ripetitivi

Scopo degli algoritmi

- Identificare un algoritmo: requisito per risolvere problemi
- Descrivere un algoritmo prima di scrivere un programma

Computational Thinking

Definizione

- **Tecniche di astrazione e risoluzione di problemi algoritmici**
- Applicabili in diverse discipline
- Combina pensiero:
    - Matematico
    - Ingegneristico
    - Problem solving

Concettualizzare

Pensare a diversi livelli di astrazione

- **problem solving**
- **Scomposizione** di problemi complessi in problemi semplici
- **Modularità** risolvere problemi a moduli riutilizzabili anche in problemi differenti

Caratteristiche dei Computational Thinkers

Competenza	Descrizione
Logica	Predizione e analisi
Valutazione	Esprimere giudizi

Algoritmi	Soluzione in passi finiti
Decomposizione	Riduzione in parti
Astrazione	Rimozione dettagli non essenziali
Creatività	Progettazione e realizzazione
Debugging	Trovare e correggere errori

Computer: Aspetti Generali

Definizione

- Sistema per **memorizzazione ed elaborazione** informazioni
- Controllato da **programmi** (lista di istruzioni)

Componenti Principali

- **Hardware**: Parte fisica (elettronica, magnetica, ottica)
- **Software**: Parte logica (dati = rappresenta qualunque tipo di informazione e programmi)

Rappresentazione Digitale

- **Bit**: Unità base di informazione
    - Due possibili valori: **0 o 1**
    - Rappresentazione fisica: on/off
- **tutta l'informazione viene rappresentata come sequenze di bit**
- **programmi:** insiemi di istruzioni (elementari) che vengono eseguite in un **ordine stabilito**

Sistemi Operativi

Sistemi Operativi: Nozioni Introduttive

Definizione di Sistema Operativo

Concetto Fondamentale

- **Collezione di programmi** che consentono l'**esecuzione delle operazioni** fondamentali del computer
- Inizializza e assicura il **corretto funzionamento** della macchina
- Prerequisito per l'esecuzione di qualsiasi altro software

Caratteristiche Principali

- **Software** che fornisce **comandi e servizi** per utilizzare al meglio la potenza di calcolo

Funzioni Principali di un Sistema Operativo

Astrazione Tecnica

- **Separazione dall'hardware: nasconde dettagli tecnici** dell'architettura hardware
- Rappresenta informazioni a un **livello comprensibile dall'utente**

Gestione Risorse

- **Coordina risorse hardware di elaborazione e memorizzazione**
- Gestisce periferiche e attività software
- Funge da **interfaccia tra utente e macchina**

Ruolo Strutturale

- Base di supporto per altri software → realizzati per essere **supportati** dal SO
- Strettamente legato al **processore**, forma la piattaforma del sistema

Procedura di Avvio (Bootstrap)

Fase Iniziale

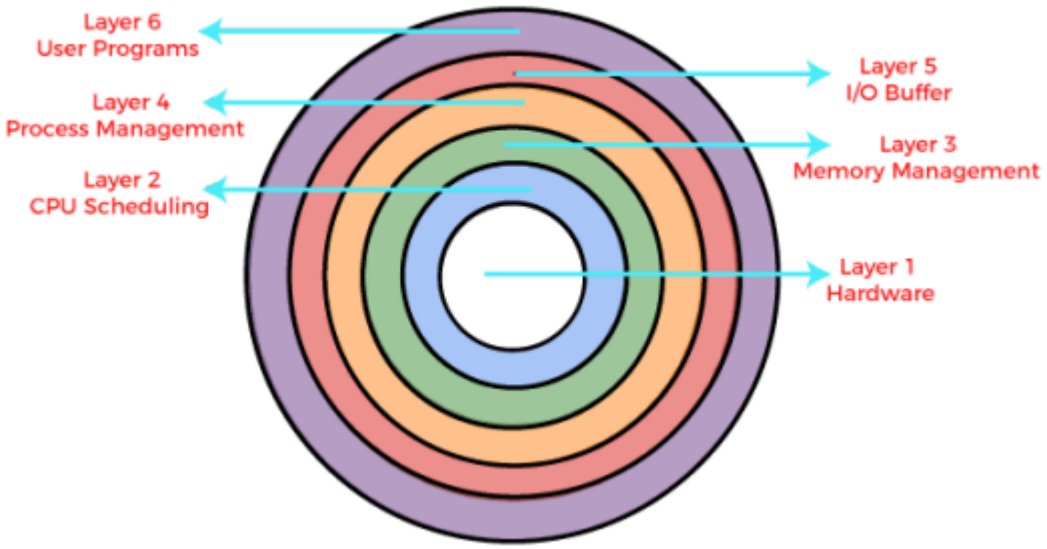
- Procedura memorizzata **nella memoria ROM**
- Quando si accende il computer, **la RAM è vuota**
- **La CPU legge le istruzioni di bootstrap dalla ROM** per inizializzare la macchina

Caricamento del Sistema Operativo

1. Ricerca e caricamento dell'OS dall'**hard disk**

2. Caricamento **nella RAM** ed esecuzione
3. Accesso al pannello di controllo (es. desktop Windows)

Livelli del Sistema Operativo



Hardware Abstraction Layer (HAL)

- **Interazione diretta con l'hardware**
- Fornisce **astrazione per i driver** di dispositivo
- Permette al **kernel** di comunicare con **componenti hardware**

Driver di Dispositivo e Gestione I/O

- Gestione di **specifici dispositivi hardware**
- Comunicazione tra **kernel e hardware** fornendo un' **interfaccia standard** per dispositivi

Gestione dei Processi

- **Creazione, pianificazione e terminazione** dei processi
- Gestione del **multitasking**
- Assegnazione **priorità e tempo CPU**

Gestione della Memoria

- Gestione **memoria di sistema**: fisica e virtuale
- **Allocazione e deallocazione**
- Paging e swapping per uso efficiente della memoria

Gestione File System

- **Lettura e scrittura file**
- **Organizzazione** e **accessibilità** dei file

Networking

- Gestione aspetti di **rete**
- Implementazione **protocolli** (es. TCP/IP)
- **Trasmissione dati**

Comunicazione Inter-Processo (IPC)

- Meccanismi di **comunicazione tra processi** → Coordinamento attività tra processi

Interfaccia Chiamate di Sistema

- **Ponte tra applicazioni utente e kernel**
- Fornisce chiamate di sistema per **richiedere servizi** *(es. operazioni su file)*

Interfaccia Utente

- Programmi per **comunicazione diretta con l'utente** *(GUI o Shell)*

Linguaggi di Controllo del Sistema Operativo

Caratteristiche

- Ogni sistema operativo ha un **proprio linguaggio**
- Comandi per eseguire e controllare operazioni

- **Sintassi** formale e rigida
- Impartiti tramite **riga di comando** o interfaccia grafica

Linux

Introduzione a Linux e UNIX

Origini di Linux

- Sviluppato nel 1991 da **Linus Torvalds**
- sistema operativo **libero**
- **Licenza GNU**: permette a chiunque di **utilizzare, modificare e condividere il codice sorgente** → vasta **collaborazione** globale tra sviluppatori.

Autenticazione

- Utenti hanno **username e password** per autenticarsi → memorizzati nel file **/etc/passwd**
- **multiutente:** più utenti nello stesso sistema contemporaneamente

Diritti di accesso

- Ogni oggetto di sistema ha specifici **diritti di accesso**
- **gruppi logici** di utenti per gestire i permessi
- **root:** utente speciale che possiede tutto il controllo sul sistema

FileSystem

- struttura (ad albero) su disco → contiene tutti i dati del S.O. e dei suoi utenti.
    - file
    - directory
- **root (/)**: directory che contiene tutti gli oggetti del filesystem

Path

- **percorso assoluto:** percorso dalla radice al file (*/home/utente/file.txt*)
- **percorso relativo:** Da una directory prefissata al file

directory	funzione
/home	personale degli utenti. file e impostazioni personali
/bin	eseguibili essenziali (comandi)
/boot	file per l'avvio del sistema (kernel e boot loader)
/dev	file di dispositivo per le periferiche
/etc	configurazione a livello di sistema per SO e applicazioni
/lib	librerie condivise
/tmp	file temporanei
/usr	utilità e applicazioni per utenti

La Shell Linux

Definizione

- interfaccia a **riga di comando:** utenti interagiscono con il sistema operativo.
- seguire comandi, gestire file e directory, e configurare il sistema.
- **Prompt dei comandi**

Comandi e Opzioni

- **Opzioni**: con "-" per indicazioni sul compito
- **Comandi Builtin**
    - cd
    - ls
    - pwd: pathname assoluto della directory corrente
    - cp
    - mv: spostare i file
    - rm
    - mkdir
    - ps
    - kill
    - cat: visualizzare il contenuto del file

- **Comandi di aiuto**
    - -h o --help: descrizione delle funzionalità del comando.
    - man nome_comando: pagina del manuale per il comando.
    - whatis nome_comando: descrizione sintetica del comando.
    - apropos stringa: Elenca tutti i comandi che contengono la stringa specificata nella loro descrizione.
- **Redirezione dell'I/O**:

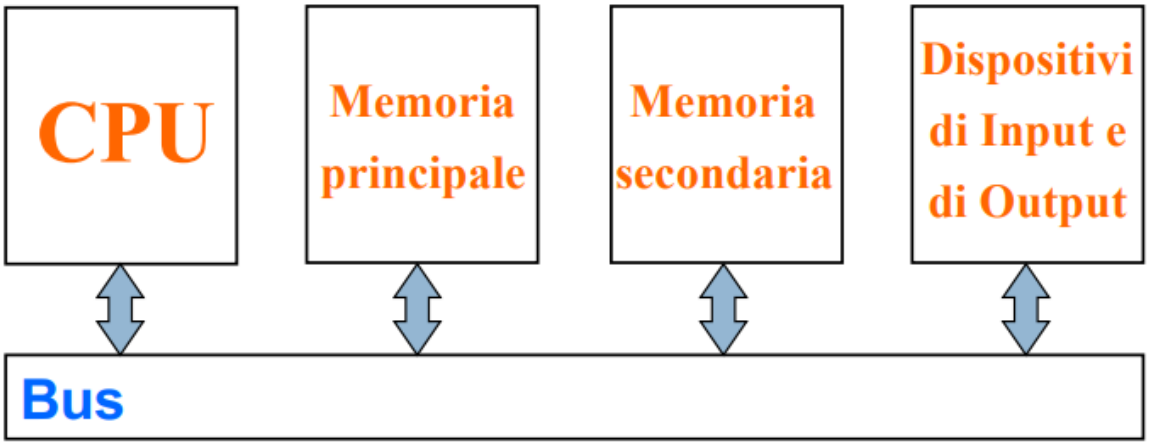
    - Ogni processo ha tre flussi principali:
        - **Standard Input (stdin)**: Input da tastiera.
        - **Standard Output (stdout)**: Output sul terminale.
        - **Standard Error (stderr)**: Messaggi di errore sul terminale.
    - **Redirezione**:
        - >: Redirige l'output su un file (sovrascrive il file esistente).
        - >>: Redirige l'output su un file in modalità append (aggiunge all'esistente).
        - <: Prende l'input da un file.
        - 2>: Redirige l'output degli errori su un file.

- **Cambio dei permessi:** chmod

- mode: indica chi può fare cosa con i permessi
    - target: a chi cambiare permessi:
        - u: user owner
        - g: group owner
        - o: other
        - a: all
    - grant:
        - +: aggiungere permessi
        - -: togliere
    - permission
        - r: read
        - w: write
        - x: execute

🏠 Architettura

Architettura di von Neumann



CPU

Descrizione

- **Funzione**: **Esegue le istruzioni** dei programmi + reperisce dati dalla memoria esterna e da periferiche e li rispedisce elaborati
- costituita da uno o più **chip**
- **Sottocomponenti**:
    - **ALU (Unità Aritmetico-Logica)**: Esegue **operazioni matematiche e logiche.**
    - **Registri**: Memoria temporanea per dati e istruzioni in uso.
    - **Unità di Controllo**: Coordina le operazioni della CPU e gestisce il flusso di dati.

Funzionamento ciclico

- **clock:** orologio di sistema, la sua **frequenza** indica quanti **cicli** vengono svolti al secondo (limitazione)
- fasi [**fetch-decode-execute**]:
    - **accesso:** lettura dell'istruzione e memorizzazione nell' **instruction register**
    - **decodifica** dell'istruzione

    - **esecuzione**
- **program counter:** contiene la **posizione dell'istruzione** che viene letta in fase di fetch.
    - **incrementato** di una posizione ad ogni ciclo → eseguite istruzioni in **sequenza** (limitazione)

parallelismo

- **descrizione:** fare più operazioni contemporaneamente → migliora le prestazioni
- **livello di istruzioni**
- **livello di processori**
    - **multiprocessore :** più processori (coordinati tra loro) con una **memoria comune**
    - **multicomputer:** più cpu con una **memoria privata** con scambio di messaggi tramite rete

Memoria

- **composizione**: suddivisa in **celle** (locazioni di memoria) con un **indirizzo** contente num predefinito di **bit**
- **Funzione**: Memorizza **dati e programmi** .

Memoria primaria:

- **veloce e costosa**
- **ROM:** sola lettura
    - **non volatile**
    - contiene i programmi per l'**avvio del computer** => sempre disponibili (BIOS)
- **RAM:** accesso casuale = <u>tempo per accedere ad un dato non dipende dalla sua posizione nella memoria</u>
    - **volatile:** perde i dati quando il computer viene spento
    - lettura e scrittura dei dati e programmi contenuti

Località spaziale

- Se si accede all'indirizzo A è molto probabile che gli **accessi successivi** richiedano celle vicine ad A

Località temporale

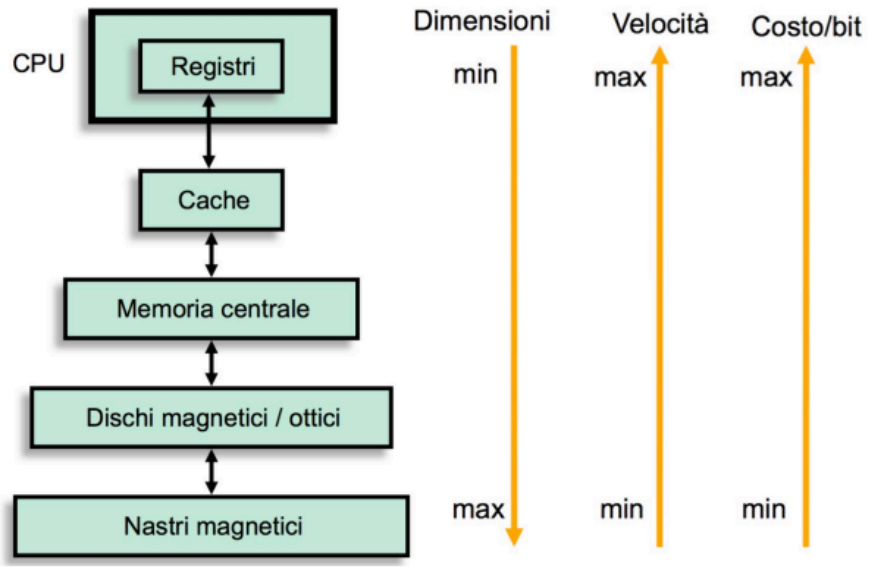
- Se si accede all'indirizzo A è molto probabile che gli accessi successivi richiedano **di nuovo l**a cella A

cache

- Memoria estremamente veloci su blocchi usati di recente

Memoria secondaria :

- **non volatile**
- **lenta** e meno costosa
- **programmi e dati**
    - **Hard disk**
    - **ssd**
    - **ottica**
    - **USB**



Dispositivi di Input/Output:

- **interazione** tra **utente** e **computer**
- **input:** tastiera, mouse, microfono
- **output:** schermo, stampanti

Bus

Definizione: Un bus è un **canale di scambio di informazioni** che collega i 4 componenti del computer.

Tipi di Bus:

- **Bus dei Dati**: Trasporta i dati **da e verso** la **CPU**
- **Bus degli Indirizzi**: solo dalla CPU. Trasporta gli indirizzi di memoria per identificare dove i dati devono essere letti o scritti.
- **Bus dei Segnali di Controllo**: solo dalla CPU. Trasmette segnali di controllo per gestire le operazioni tra i componenti.

scheda madre

- contiene la CPU, memoria primaria, bus e slot di espansione

⌨️ PROGRAMMAZIONE

Introduzione alla programmazione

Programmazione

Descrizione

- **definizione, progettazione e codifica** di programmi per calcolare
- sequenze di istruzioni che il computer esegue e di decisioni che il computer prende per svolgere attività
- corretto
    - sintatticamente
    - semanticamente
    - della soluzione
- **Bohm-Jacopini**
    - **istruzioni imperative:**
    - **istruzioni condizionali**
    - **cicli**

Linguaggi a basso livello

- **istruzioni macchina:** istruzioni elementari eseguite da un computer
    - **trasferimento dati** tra registri e memoria principale
    - **operazioni aritmetiche e logiche**
    - **salti**
- **instruction set** insieme di istruzioni macchina **specifico** per la particolare CPU
- codifica => **configurazioni di bit**
- <u>necessario scrivere all'interno della memoria primaria le configurazioni di bit corrispondenti alle istruzioni macchina</u>

Assemblatore

- scrivere codice con nomi abbreviati → **codici mnemonici (linguaggi assembly)**
- **traduce** il programma in **configurazioni di bit**

- dare nomi a indirizzi di memoria e ai valori numerici

Linguaggi ad alto livello

- operazioni scritte dal programmatore senza scendere al linguaggio macchina
- **hanno grammatica**
    - **regole lessicali**
    - **regole sintattiche**
    - **regole semantiche**

Linguaggi compilati

- **compilatore:** legge un programma in linguaggio ad alto livello => genera il programma (in un file) nel linguaggio macchina della CPU
- linguaggio compilato **dipende** dalla CPU
- C
- efficiente → traduzione viene svolta una sola volta

Linguaggi interpretati

- **interprete:** traduce il codice sorgente **senza creare un file eseguibile**
- PYTHON
- **portabile**

JAVA

---

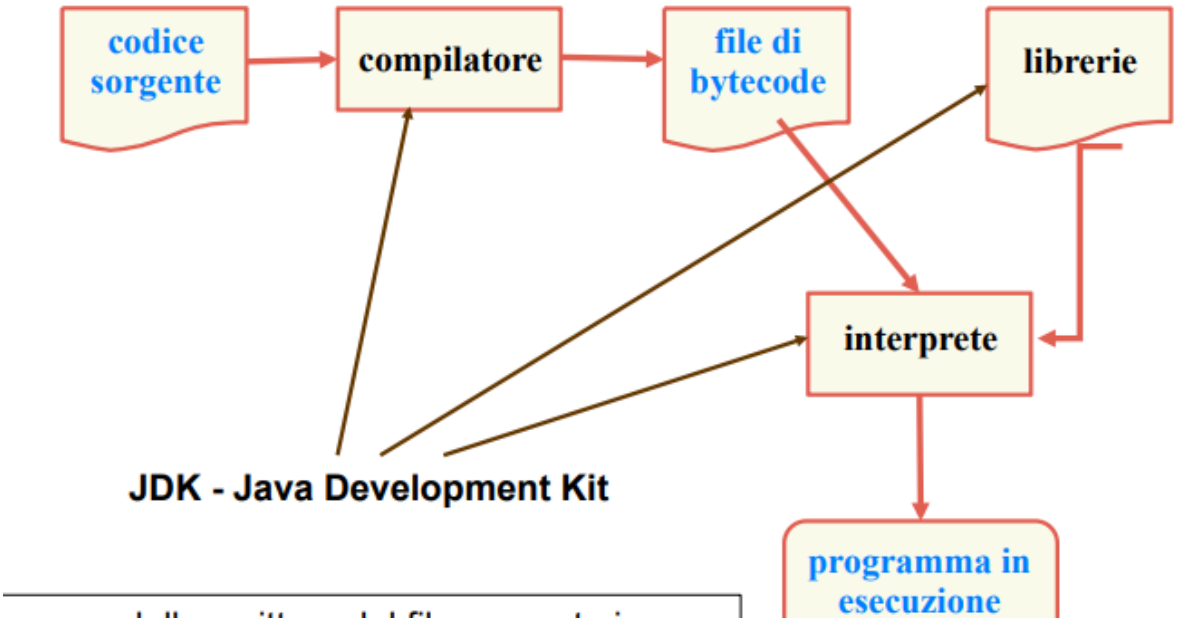
scrivere e compilare una volta ed eseguire ovunque
- compilatore crea un **bytecode** (codice eseguibile) che poi viene interpretato

- **ricca libreria standard:** es. interagire con sistema operativo

JVM - Macchina virtuale JAVA

- programma che esegue altri programmi

- eseguire programmi Java su qualsiasi dispositivo o Sistema Operativo
- comando **javac** genera un file che contiene il bytecode



- efficienza intermedia: alcune parti vengono tradotte una volta altre tradotte ad ogni esecuzione

Primo programma

Classe

- **contenitori o fabbriche di oggetti**
- ciascun file sorgente può contenere **una sola classe pubblica** con nome uguale a qualla del file

Metodi

- sequenza di istruzioni o enunciati che descrive come svolgere un metodo
- **main**
    - **static:** metodo non esamina ne modifica gli oggetti della classe

🔢 RAPPRESENTAZIONE DATI

Rappresentazione dei dati

Obiettivi di apprendimento

- **Numeri naturali**
  - Rappresentazione posizionale
- **Numeri interi**
  - Rappresentazione Modulo/Segno
  - Rappresentazione in complemento a 2
- **Numeri reali**
  - Rappresentazione in virgola fissa
  - Rappresentazione in virgola mobile
- **Caratteri**
  - ASCII
  - Unicode

Importanza della rappresentazione dei dati

- I computer utilizzano una **rappresentazione binaria** (0-1) → rappresenta quantità **finita** di numeri
  - massimo/minimo valore
  - limitazione di precisione.
- È necessaria una tabella di conversione per i caratteri

Notazione posizionale

- numeri i in **base dieci.**
- Cifre uguali in posizioni diverse hanno valori differenti, a seconda della loro posizione (peso).
- Il peso = base elevata alla potenza della posizione.

$$434 = 4 \cdot 10^2 + 3 \cdot 10^1 + 4 \cdot 10^0$$

Notazione binaria

I computer utilizzano la notazione binaria:

- numeri in base due (0 e 1)
- più facile da manipolare per i computer per **motivi tecnologici**
    - **circuiti logici** distinguono solo i livelli **"acceso"** e **"spento"**
- $[0, 2^n - 1] \cap Z$ numeri rappresentabili

Rappresentazione in virgola fissa

---

parte frazionaria

1. **moltiplico** per 2 il numero
2. si **sottrae 1** se il numero è magg/uguale a 1
3. continuo fino a che il risultato è uguale a **0** oppure è un risultato **già ottenuto**

- Si riscrive il numero dalla prima cifra intera ottenuta, periodico se si ripete il risultato

Numero in virgola fissa

numero si ottiene componendo la parte intera e quella frazionaria e la virgola rimane invariata di posizione

Numeri negativi

Modulo e segno

- Il **primo bit** rappresenta il segno (0 = positivo, 1 = negativo).
- I restanti bit rappresentano il **valore assoluto** del numero.

$[-(2^{n-1}-1), 2^{n-1}-1] \cap Z$

- 0 ha **due rappresentazioni**
- **algoritmo complesso per la somma di numeri**

Complemento a due

$[-2^{n-1}, 2^{n-1}-1] \cap Z$

- numero **positivo** => normale rappresentazione binaria.
- numero **negativo** =>
  - appresentazione del suo valore positivo, si inverte i bit e si aggiunge 1.
  - sommo 2^n e converto
- bit più significativo è il **segno**

errore di overflow: quando si sommano due numeri con lo stesso segno e il risultato è di segno opposto → PK, supero i limiti della rappresentazione disponibile.

Problemi

- **Numero fissato di bit** per parte intera e parte frazionaria
- numeri rappresentati in modo **approssimato**

Numeri in Virgola Mobile

- notazione in base due con **segno, mantissa** ed **esponente**.
- **spostare la virgola (**moltiplicazione**)**
- prima cifra significativa = a sinistra del punto decimale.

Standard IEEE 754

- definisce il formato e la disposizione dei bit
  - 32 bit (singola precisione)

- 64 bit (doppia precisione).

1 bit	8 bit	23 bit
Segno	Esponente con bias $e + 127$	Mantissa (senza 1 iniziale)

Singola precisione

1 bit	11 bit	52 bit
Segno	Esponente con bias $e + 1023$	Mantissa (senza 1 iniziale)

Numeri Speciali

- **Zero**: esponente -127, mantissa di 0.
- **Infinito**: esponente +128 mantissa di 0.
- **NaN** (Not a Number): esponente di +128 e una mantissa diversa da 0

Esponente e Mantissa

- esponente: **l'intervallo di potenze di 2** in cui il numero è definito
- mantissa indica la porzione di quell'intervallo.

31	30							23	22																						0
0	1	0	0	0	0	0	0	0	1	0	0	1	0	0	0	1	1	1	1	0	1	0	1	1	1	0	0	0	0	1	1

$(10000000)_2 = 128$
Tolgo il bias: 128-127 = 1
E=1

$2^{-1}+2^{-4}+2^{-8}+2^{-9}+2^{-10}+2^{-11}+2^{-13}+2^{-15}+2^{-16}+2^{-17}+2^{-22}+2^{-23} =$

Oppure

$(2^{22}+2^{19}+2^{15}+2^{14}+2^{13}+2^{12}+2^{10}+2^{8}+2^{7}+2^{6}+2^{1}+2^{0}) \times 2^{-23} =$

M = 0.57000005460003

Valore: segno $1.M \times 2^E = + \ 1.57000005460003 * 2 = 3.1400001092$

Limiti della Rappresentazione

- Il numero **più piccolo** rappresentabile: **1.8×10^{-38}**
- numero più grande: **3.4×10^{38}**

- La distanza tra due numeri reali successivi : $\mathbf{2^{-23} \times 2^{E}}$

Errori di Arrotondamento

- I numeri non vengono rappresentati con un'approssimazione corretta
- errore si amplifica moltiplicando il numero
- **errori nelle somme**

Rappresentazione Esadecimale

- 16 simboli (0-9, A-F).
- Conversione da binario a esadecimale → Raggruppare i bit a gruppi di **quattro**.

Rappresentazione Ottale

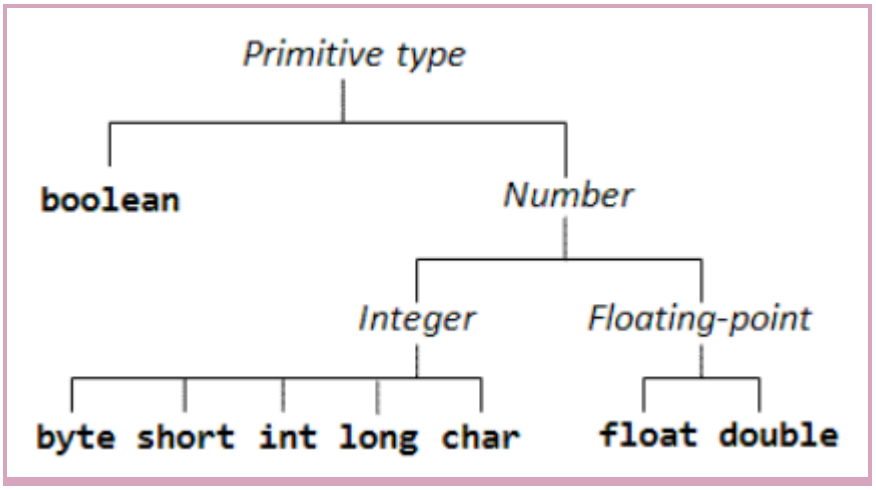
- Conversione da binario a ottale → Raggruppare i bit a gruppi di **tre**.

Rappresentazione di Caratteri

- I caratteri come **numeri naturali** (senza segno).
- Codice **UNICODE**:
  - **2 byte** per carattere.
  - 2^{16} =**65536 caratteri**
  - I primi **128** = l'ASCII.
- Codice **ASCII**:
  - 7 bit per carattere → **128** caratteri.
  - I primi **32** = caratteri di controllo (es. tabulatore, nuova riga).
  - **ASCII esteso:** 8 bit per 256 caratteri (inclusi caratteri speciali).

Tipi di Dato in Java

- Java: **fortemente tipizzato** = Alla compilazione ogni dato deve essere di un **tipo preciso**
  - **Primitivi**:



- **Riferimenti**: puntano a oggetti.

Tipi Primitivi

- **byte**: 1 byte, valori da -128 a 127.
- **short**: 2 byte, valori da -32768 a 32767.
- **int**: 4 byte, valori da -2^{31} a 2^{31-1}.
- **long**: 8 byte, valori da -2^{63} a $2^{63}-1$.
- **float**: 4 byte, numeri in virgola mobile ($\pm 10^{38}$, 7 cifre significative).
- **double**: 8 byte, numeri in virgola mobile (±10^308, 15 cifre significative).
- **boolean**: true o false.
- **char**: 2 byte, valori da 0 a 65535 (Unicode).

- pacchetto **java.lang** contiene una classe per ogni tipo di dato fondamentali
  - definita la costante MAX_VALUE e MIN_VALIE

Attenzione!

- In Java, i tipi **interi** usano **complemento a due.**
- La JVM non segnala overflow nelle operazioni aritmetiche; si ottiene un risultato errato.
- Divisione per zero genera un'eccezione **ArithmeticException** per interi, mentre per float/double si ottiene **+/- Infinity.**
- per i long e float è necessario aggiungere rispettivamente il carattere L e f al termine del numero

Variabili e Costanti

- Ogni **variabile** ha:
  - **Nome**: identificativo.
  - **Tipo**: determina i valori **ammissibili** e l'**occupazione di memoria.**
  - **Contenuto**: valore associato.
- **Costanti**:
  - Non viene **modificato mai più** dopo l'assegnazione .

Literals, Promozioni e Casting

- **Literals**: valori costanti interpretati come tipi **int** e **double**.
- **Promozioni**: **conversione automatica** quando si assegna un valore a un tipo **più capace.**
- **Casting**: **conversione forzata** per assegnare un valore a un tipo meno capace.
  - senza => **errore in compilazione** pk potrebbe portare alla perdita di cifre significative

Esempi di Literals

- Interi:
  - byte value1 = 10; // OK
  - short value3 = 200; // OK
- Float:
  - float f2 = 2.35f; // OK

Esempi di Casting

- `int i = 100; byte b = (byte)i; // OK!`
- `int xInt; double xDouble = 223.8644; xInt = (int)xDouble; // parte decimale troncata`

programmazione base

Terminologia di Base per la Programmazione

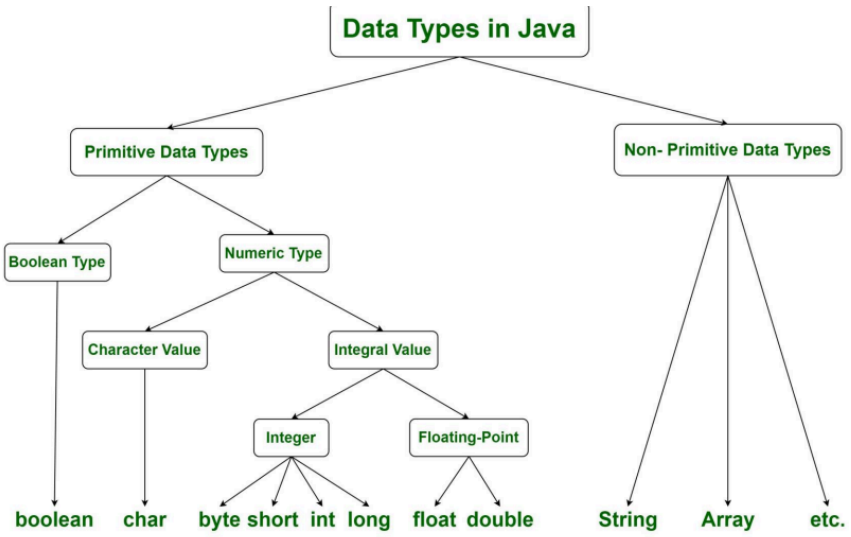
Variabili

- Definizione: Spazi di memoria identificati da un **nome** → conservano **valori** di un determinato **tipo**
- Caratteristiche:
  - Nome: per "usare" la variabile
  - Tipo
  - Contenuto
  - **Attributi** (visibilità, staticità)
  - **Scope** (ambito di accesso)

Perché usare diverse variabili

- programma **comprensibile** -> capire lo scopo
- diversi spazi di **memoria** e tempi di **elaborazione**

Tipi di Dati in Java



Uso delle Variabili

- **Dichiarazione**: Specifica nome e tipo.
- **Assegnazione**: Cambia il valore della variabile.
- **Definizione**: Dichiarazione e assegnazione iniziale → fissa nome, tipo e attributo + **visibilità**

Una variabile può essere definita una volta sola, MA l'assegnazione può avvenire più volte

Nomi delle Variabili

- **variabili d'istanza:** associate ad un oggetto e accessibili in tutta la classe. esistono fino a quando esiste l'oggetto
- **variabili statiche:** associate alla classe stessa e condivise a **tutte** le istanze della classe
- **variabili locali:** definite dentro metodi o blocchi → esistono solo nel metodo/blocco
- **parametri:** definite nelle firme dei metodi

Trasferimento dati

- **lettura dei dati**
  - non sono **distruttive** => le informazioni rimangono invariate
- **scrittura dei dati**
  - **distruttiva =>** sovrascrivo le informazioni alla destinazione

ATTENZIONE: ASSEGNAZIONE

Se **non** avviene **l'assegnazione:**

- creata la variabile
- **NON allocato spazio** in memoria
- non ha **nessun valore**
- **Non leggibile**
- **ERRORE IN COMPILAZIONE**

Commenti

- scopo
    - **chiarezza**
    - **manutenibilità:** capire il codice dopo tempo
    - **collaborazione**
    - **apprendimento**

Costanti

- **Definizione**: Valori che **non modificabili (**altrimenti => ERRORE SEMANTICO segnalato dal compilatore **)**
- Sintassi: final nomeTipo NOME_COSTANTE = espressione;
- Convenzione: Nomi in maiuscolo.

Operatori numerici

- **punto** per separatore decimale
- notazione esponenziale *es. 1.93E3*

Metodi

- Classe Math: **Metodi statici** per operazioni matematiche
- Sintassi di Invocazione **metodi statici:** **NomeClasse.nomeMetodo(parametri).**
    - invocare un metodo definito nella classe NomeClasse

    - non possono interagire con le variabili d'istanza ma solo quelle statiche
    - associato alla classe → esiste una sola "copia"
- Sintassi di Invocazione **metodi NON statici:** **NomeClasse.nomeOggetto.nomeMetodo(parametri).**
    - agisce su un oggetto
    - associati ad ogni singola istanza

esempi

```
public class Geometry2DUtils {
    public static final double PiGreco = 3.141592;
    public static double areaParallelogramma(double base, double altezza) { }
    public static double areaParallelogramma(double c1, double c2, double alpha)
    // ...
```

tutte le classi possono usarlo, indipendentemente dalla specifica istanza

```
public class Parallelogramma {
    private double base, altezza;
    public Parallelogramma(double base, double altezza) {
        // inzializza le variabili di istanza
        this.base = base;
        this.altezza = altezza;
    }
    // questo metodo area non ha bisogno dei
    // parametri in quanto utilizza le variabili della classe
    public double area() {
        // utilizza il metodo (pubblico e statico) nella classe  di utility
        // chiamandolo usando il nome della classe (poi il compilatore
        // sceglie quello giusto sulla base dei parametri)
        return Geometry2DUtils.areaParallelogramma(base, altezza);
    }
    // metodi per l'accesso (getters) delle variabili private della classe,
    // garantiscono, tra l'altro, che i field possano essere utilizzati ma n
    // modificati
    public double getBase()    { return base; }
    public double getAltezza() { return altezza; }
}
```

area() si riferisce ad ogni singola istanza

Libreria Standard

- **JDK** fornisce la **documentazione API** per la descrizione delle classi della libreria Standard

Pacchetti

- Tutte le classi della libreria standard sono raccolte in **pacchetti (package)**
- **importare** la classe della libreria per usarla
    - **java.lang** viene importato automaticamente

🅰️ Stringhe

Stringhe e Input in Java

Introduzione alle Stringhe

- **Definizione**: Sequenza di caratteri racchiusi tra virgolette.
- sono **oggetti => non tipi primitivi**
- con una stringa si possono **invocare metodi** (**non statici**) della classe String
    - length: non statico -> su un oggetto
- concatenazione con "+" → variabile non stringa viene **convertita** in stringa
- **Immutabilità**: non modificabili dopo la creazione → i metodi che agiscono sulla stringa ne **ritornano** una nuova

Metodi Utili della Classe String

- Conversione a Maiuscolo e Minuscolo:
    - toUpperCase(), toLowerCase().
- **substring**: se parametro non valido → **errore in esecuzione** .
- toString(): per convertire int o double a Stringhe (Integer.toString(numero))

Caratteri in una Stringa

- Tipo **char**: Per memorizzare singoli caratteri.
- char = tipo di dato **fondamentale (non è una classe)**
- occupa meno spazio in memoria rispetto una stringa

Sequenze di Escape

- Uso del **Backslash** (\): Per inserire caratteri speciali.
- Nuova Riga: \n per andare a capo.

- Per inserire \ nella stringa uso \\
- per inserire caratteri non presenti nella tastiera => codifica standard Unicode

Dati in input

- interprete java mette a disposizione il proprio **input standard** (flusso di input) con **System.in**
- Classe **Scanner** (in java.util): Per leggere input da tastiera.
    - creare un **oggetto** della classe **Scanner**
- Metodi di Lettura:
    - nextInt(), nextDouble(), nextLine(), next().
    - il programma si **ferma** per attendere l'input

👷🏻 COSTRUTTORI

Costruttori

Introduzione ai Costruttori

- **Definizione**: Metodi speciali per **inizializzare oggetti** (esemplari) di una classe e il loro **stato**
- Nome: Deve corrispondere al nome della classe (iniziale maiuscola).
- Restituzione: **Non restituiscono valori,** nemmeno void.
- scopo: **inizializzare le variabili di esemplare** di un oggetto
- solitamente **public**

Invocazione dei Costruttori

- Operatore **new**: per creare un nuovo oggetto.
  - new predispone la memoria → **riferimento** all'oggetto appena creato
  - costruttore definisce lo stato iniziale
- **se non viene definito nessun costruttore =>** compilatore si comporta come se fosse presente un **compilatore predefinito**

Inizializzazione delle Variabili

- variabili esemplare non inizializzate subito → **valori Predefiniti**:
  - Numerici: 0
  - Booleani: false
  - Oggetti: **null** → riferimento a **nessun oggetto** → **NON può invocare metodi** → **nullpointerexception**
- L'inizializzazione nel costruttore prevale l'inizializzazione predefinita

Costruttore predefinito

- **se non viene definito nessun costruttore =>** compilatore si comporta come se fosse presente un **costruttore predefinito (IMPLICITO)**
- **pubblico**
- **nessun parametro**
- esiste solo se non sono definiti altri costruttori
- valori iniziali assumono valori predefiniti (vedi sopra)
- non funziona **new**

Costruttori Sovraccarichi - overloading

- Definizione: Una classe può avere **più costruttori** con lo stesso nome ma **parametri diversi**.
- Compilatore decide quale costruttore invocare in base al numero e al tipo di parametri
    - se non trova = errore

Commentare l'interfaccia pubblica

- **Javadoc** genera **automaticamente** la documentazione in formato **HTML** (/** JavaDoc commento */)
- ciascun metodo/costruttore ha un commento
    - @param per i parametri
    - @return per il valore restituito
    - @throws eccezione lanciata
    - @author nome dell'autore
    - @version

🏫 CLASSI

Programmazione ad Oggetti

Utilizzo di Classi e Oggetti

- Elaborazione di **dati complessi => oggetti**
- **Oggetti** e il loro **comportamento**: Descritti da classi e metodi.

Classi

- Definizione: **Fabbrica di oggetti,** specifica metodi e dati.
- Contenuto:
    - **metodi e dati** la cui realizzazione non è necessariamente nota.
    - **Interfaccia pubblica** della classe: permette l'accesso dei dati => specifica i metodi invocabili attraverso gli oggetti di quella classe (istanze)

Oggetti

- entità manipolata attraverso l'invocazione dei metodi
    - **metodo** = accedono ai dati interni dell'oggetto
- funzionamento come **black box**
    - **interfaccia pubblica =** insieme dei metodi che definiscono il comportamento dell'oggetto
    - **implementazione "nascosta"**
- **oggetto = esemplare di una classe**

Obiettivi

- Utilizzo di oggetti: Creazione e invocazione di metodi da classi esistenti (es. classe String).
- Distinzione: Usare oggetti vs. realizzare una classe.

Definizione e Inizializzazione di Variabili Oggetto

1. **creare** un oggetto e **inizializzarlo**
2. posso **chiamare metodi** attraverso la **variabile oggetto** di tipo "NomeClasse"

Metodi

- Istruzioni valide:
    - **Assegnazione valori** restituiti dai metodi a **variabili compatibili.**
    - Uso di valori restituiti come **parametri** per altri metodi.
- Istruzioni non valide: Invocazione di metodi **non disponibili per il tipo di oggetto.**

Metodi: Parametri Espliciti/Impliciti

- **Parametri espliciti**: Specificati tra parentesi, valori che **specificano i dati da elaborare**
- **Parametri impliciti**: Informazioni memorizzate nell'oggetto invocante (es. ogg.length())

Metodi: return

- termina l'esecuzione del metodo **tornando all'esecuzione sospesa d**el metodo invocante
- Se contiene un espressione => valore restituito al metodo invocante
- Se metodo è void => il return è **implicito**

Metodi: this

- riferimento all'oggetto con il quale è eseguito il metodo (parametro **implicito**)

```
account.deposit(500);
————————————
public void deposit(double amount) {
    this.balance = this.balance + amount;
}
```

- ATTENZIONE: i metodi statici **non** hanno il parametro implicito

- non hanno accesso alle variabili di esemplare della classe su cui sono definiti

Definizioni di Metodi: Firma

- Contenuto della firma:
  - Specificatore di **accesso** (
    - **public**: può essere invocato da qualsiasi metodo di qualsiasi classe
    - **private** .
  - Tipo di dato **restituito**.
  - **Nome** del metodo.
  - Elenco di **parametri**.

ATTENZIONE return e throw

- Se in un metodo viene invocato **return** o **throw** all'interno di un if => clausole **else è inutile**

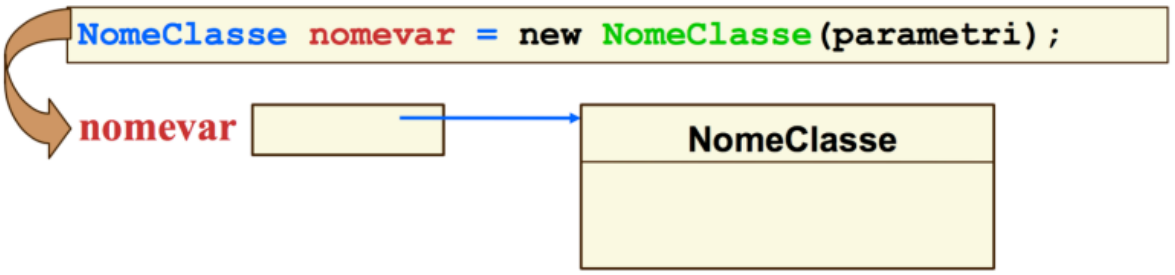
```
 if (n < 0){
   throw new IllegalArgumentException();
 } else if (n == 0){
    return 1;
 }else {
   int p = 1;
    for (int i = 2; i <= n; i++){
    p = p * i;
  }
  return p;
}
```

Else inutili:

- se viene eseguito il **throw** → codice dopo non viene eseguito in ogni caso
- eseguito **return** → codice dopo non viene eseguito in ogni caso

Variabili Oggetto

- conserva il **riferimento** all'oggetto NON l'oggetto
    - **Riferimento**: informazioni sulla **posizione** in memoria.
- **ATTENZIONE**: Dichiarazione di un oggetto senza assegnazione => riferimento a nessun oggetto
- creazione con **new**
    - crea l'oggetto e restituisce il riferimento
    - seguito dal costruttore (NON è UN METODO)



Accorgimenti

- **Re-inizializzare** un oggetto: creare un nuovo oggetto (new NomeClasse()) e assegnare il riferimento all'oggetto da re-inizializzare

```
Rectangle box = new Rectangle(5,10,20,30);
box = new Rectangle(20,35,20,20);
```

Copiare oggetti

- ATTENZIONE: **Assegnazione diretta copia il riferimento, non l'oggetto.**
- **Copiare oggetti**: Creare un **nuovo oggetto** per avere una **copia indipendente.**

```
Rectangle box = new Rectangle(5, 10, 20, 30);
Rectangle box2 = new Rectangle(box);
```

Classe BankAccount

- Metodi:
  - **getBalance**(): Restituisce il saldo.
  - **withdraw(double amount):** Preleva l'importo.
  - **deposit(double amount):** Deposita l'importo.

Incapsulamento

- **Definizione**: Variabili di esemplare private, accessibili solo tramite metodi pubblici.
- Vantaggi:
  - **Controllo sull'accesso** allo stato dell'oggetto.
  - Modifiche interne senza impatti esterni.
  - programmatore ignaro dell'implementazione del metodo che usa

```
double b = account.balance;  NO -> balance è privato
double b = account.getBalance();
```

Metodi di Accesso

- Convenzione: Metodi getNomeVariabile() e setNomeVariabile(...) per gestire **l'accesso alle variabili di esemplare.**

❓ DECISIONI

Decisioni

Alterare il flusso di esecuzione in base a **condizioni**.

enunciato if

- **Capacità di prendere decisioni.**
- Composizione:
  - **Verifica di condizione.**
  - **Corpo** eseguito se e solo se la condizione è **vera**.

Clausola else

- Utilizzo per gestire **alternative**.
- Vantaggio: **una sola verifica condizionale** => se modifico la condizione non è necessario modificare codice (es. altro if al posto di else)
- appartiene sempre all'if più vicino, NON per tabulazione (se non ci sono parentesi)

Alternative multiple

- Utilizzo di sequenze di confronti con else if.
- **Importanza dell'ordine nelle condizioni**.
  - *es. confronti con maggiore => valori più alti sopra*

Confronto

- **Operatori relazionali**: >, <, >=, <=, ==, !=.
- Uso di **equals()** per confrontare stringhe.
  - altrimenti (con =) confronto il **riferimento all'oggetto Stringa**
- **equals()** da sovrascrivere per gli oggetti => confronto delle variabili d'oggetto

Confronti in virgola mobile

- ATTENZIONE: i numeri hanno precisione limitata => **errori di arrotondamento**
    - necessario prevedere una **tolleranza**

```
final double EPSILON = 1E-14;
double xyMax = Math.max(Math.abs(x),Math.abs(y));
 return Math.abs(x - y) <= EPSILON * xyMax;
```

Ordinamento lessicografico

```
s1.compareTo(s2)
```

- Utilizzo di **compareTo()** per confrontare stringhe. Restituisce int
    - **negativo** se parametro implicito (s1) **precede** esplicito
    - **positivo** se parametro implicito **segue** esplicito
    - **zero** se sono identiche
- compareTo confronta carattere per carattere le stringhe
    - numeri precedono le lettere
    - Maiuscole precedono minuscole
    - spazio precede tutti
- compareTo per gli oggetti => da definire per la classe

Operatori booleani

- svolgere operazioni su **boolean (tipi fondamentali)**
- Utilizzo di variabili booleane e operatori logici (&&, ||, !).
- Leggi di **De Morgan** per semplificare espressioni logiche.
- precedenza al **not** → **and** → **or**

🔃 ITERAZIONI

Iterazioni

Cicli in Programmazione

Ciclo while

- Utilizzo: Eseguire un blocco di istruzioni finché la condizione è **vera**.
- sintassi: while(condizione) {enunciato}.
- ATTENZIONE: alcuni errori **logici** impediscono la terminazione del ciclo => **ciclo infinito**
    - *es. non modificare nel corpo la variabile (o altro) che appare nella verifica => necessario arrestare il programma con **CTRL-C***

Ciclo for

- Sintassi: for(inizializzazione; condizione; aggiornamento){enunciati}
    - Inizializzazione può definire una variabile **visibile solo all'interno** del corpo del for
- Utilizzo: Comodo per cicli con un **numero definito di iterazioni.**
- **visibilità variabili:** se un valore deve essere visibile anche fuori dal ciclo => definirla prima (*es. anche il contatore*)

Ciclo do-while

- per **"ciclo e mezzo":** "fai qualcosa, verifica, fai altro e ripeti se la condizione è vera"
- utilizzo: per eseguire il corpo del ciclo **almeno una volta**
- sintassi: do{ enunciato} while(condizione);

Break

- per terminare un ciclo => l'esecuzione in corso non viene completata e il ciclo termina
- Uso anche per verificare una condizione all'interno del ciclo
-

```
while(true){
  if(condizione){
      break;
   }
   altro codice
}
```

continue

- per terminare l'**iterazione corrente** => passaggio alla successiva
- Uso anche per verificare una condizione all'interno del ciclo

Switch

- svolgere una sequenza di confronti su un'unica variabile
- vantaggi
    - non ripetere il nome della variabile nei confronti
- svantaggi
    - solo se la variabile da confrontare è **intera** o **stringa**
    - NO se il valore non è costante
    - ogni case ha un break => altrimenti eseguo tutti i case
-

```
int x;
switch (x) {
    case 1:
        ...
        break;
    case 2:
        .....
        break;
    default:
        .....;
```

```
        break;
}
```

Gestione dell'Input

- Elaborare n dati con **n conosciuto a priori**
    1. acquisire n da input
    2. usare n nella condizione di un ciclo
- Elaborare n dati con **n NON noto**
    - **Sentinella**
    - **segnale di fine inserimento**

Input con Sentinella

- Uso: quando devo leggere una sequenza di dati in ingresso finché non sono finiti.
- strategia: usare un carattere **non valido** per segnare la fine dell'acquisizione di dati
    - dato **non** appartenente al **dominio di elaborazione** del problema: *es. inserisco un numero negativo per terminare una sequenza di dati positivi*
    - inserire un dato di **tipo diverso:** *es. inserire una stringa al posto di un double*

Input con Scanner

- strategia: inserire dati finché non finiscono
- Metodi **predicativi**:
    - **hasNext(), hasNextLine(), hasNextInt(), hasNextDouble().**
    - restituiscono **false** se i dati sono finiti o di tipo diverso da quello specificato nei metodi
    - ATTENZIONE: non consumano i dati => posso recuperarli (dal buffer) nelle istruzione successive con next

- Terminazione: Utilizzare **Ctrl+Z** (Windows) o **Ctrl+D** (Unix) per segnalare la fine dell'input.

ARRAY

Array

Definizione di Array

- Definizione: Un array è una collezione di dimensione arbitraria di elementi dello **stesso tipo,** accessibili tramite **indice**.
- **sequenza posizionale di dati omogenei**
- **ad accesso casuale:** il tempo di accesso (lettura e scrittura) **non** dipende dal valore dell'indice associato all'elemento

Strutture dati - intro

- strumenti per organizzare l'info
  - **struttura dati astratta (ADT)** descrive i dati e le funzioni che la struttura deve avere
  - **struttura dati:** implementa l'ADT
- ADT Sequenza
  - lineare: ogni elemento ha un predecessore e successore
  - operazioni: inserimento, rimozione, ricerca
- Strutture dati che implementano ADT
  - Array
  - Lista concatenata

Creazione

- Un array è un **oggetto**
  - ATTENZIONE: una variabile che si riferisce ad un array è una variabile oggetto che contiene il riferimento all'oggetto array
- tipo di dati: qualsiasi
- dati possono essere **duplicati** in posizioni diverse
- Alla creazione: tutti gli elementi vengono **inizializzat**i al loro **valore di default**

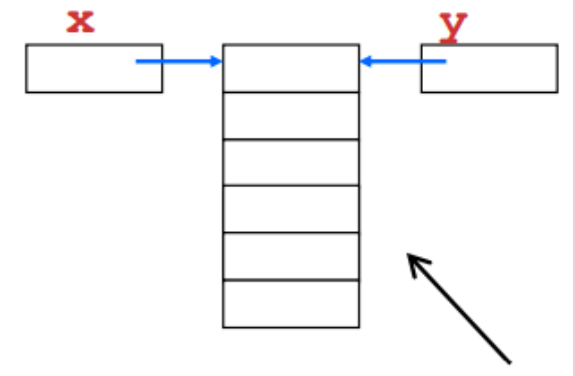
Dimensione e caratteristiche

- si può conoscere attraverso la sua **variabile pubblica di esemplare length**
    - variabile di tipo **final =>** non modificabile (per questo va bene che sia pubblica)
- Possono essere "passati" come **parametri espliciti** di metodi **statici**

Copiare o clonare un array

- **copiare:** creare un altro riferimento allo stesso array

```
double[] x = new double[6]
double [] y = x
```



- **clonare:** ottenere una copia dell'array
    1. creare un nuovo array dello stesso tipo e dimensione
    2. copiare gli elementi nella stessa posizione
    - oppure metodo statico **arraycopy** della classe **System**

```
double[] values = new double[10];
double[] otherValues = new double[values.length];
for (int i = 0; i < values.length; i++){
        otherValues[i] = values[i];
}
```

Errori Comuni Negli Array

- **ArrayIndexOutOfBoundsException**: Accesso a un **indice al di fuori del range** consentito.
- Inizializzazione Non Corretta: Tentativo di accesso a elementi non inizializzati.
- **NegativeArraySizeException** se la dimensione è negativa

Array Riempiti in parte

Problema

- Creare un array la cui **dimensione non si conosce a** priori.
- MA: array hanno dimensione **fissa**.

Soluzione - array riempito in parte

1. Definire un array di **dimensioni ragionevoli**.
2. Usare una **variabile** per tracciare **l'ultima posizione valida**:
    a. Aggiungere elementi fino a **riempire** l'array o **terminare la lettura.**
- **NON è un tipo primitivo**

Dimensione Logica e Fisica

- **Dimensione fisica**: Lunghezza **massima** dell'array (`values.length`).
- **Dimensione logica**: **Numero di elementi** effettivamente memorizzati

Prevenire Errori di Overflow

- **Verificare se l'array è pieno** prima di aggiungere nuovi elementi.
    - ArrayIndexOutOfBoundsException se si raggiunge il limite

Ridimensionare un Array (Array Dinamico)

1. Creare un **nuovo array** più grande.
2. **Copiare i dati** dall'array originale.
3. **Rimpiazzare** il riferimento al vecchio array.

```
if (valuesSize == values.length) {
   double[] newValues = new double[values.length * 2];
   for (int i = 0; i < values.length; i++) {
      newValues[i] = values[i];
   }
   values = newValues; // Il vecchio array viene "abbandonato".
}
```

String[] args

- parametri che vengono passati da riga di comando quando si esegue un programma
    - java Programma <parametri>
- letti dall'**interprete** e trasformati in **array di stringhe** passati come parametro al main
    - stringhe di più parole => dentro “”
- lunghezza = num parametri forniti

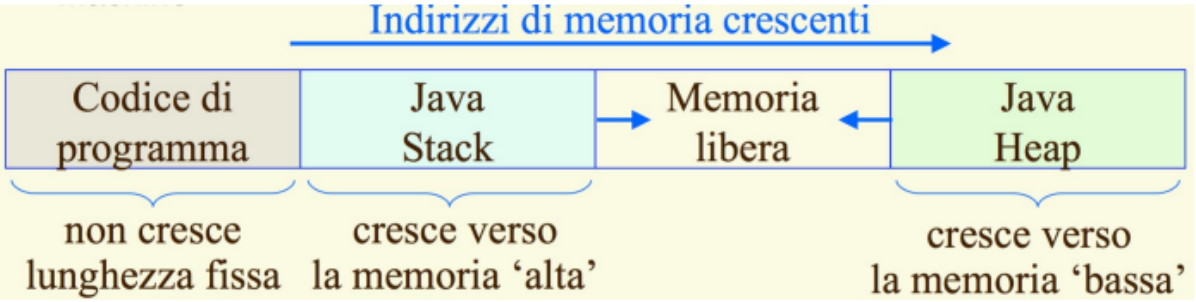
Gestione della Memoria

Garbage Collection

- Processo **automatico** nella **JVM** che libera la memoria occupata da oggetti **non più referenziati =>** memoria riutilizzabile

Allocazione della memoria

- All'esecuzione del programma => assegnata un'area di memoria al programma
    - parte **statica:** per il codice (le sue dimensioni sono fisse durante l'esecuzione)
    - **Java Stack:** dinamica per salvare parametri e variabili locali dei metodi
    - **Java Heap:** dinamica per gli oggetti (creati con **new**)



Array Bidimensionali e Multidimensionali

Definizione

- Una **matrice** o **array bidimensionale** è una struttura dati organizzata in righe e colonne.
- Gli elementi sono identificati da una **coppia ordinata di indici**:
  - Indice **di riga.**
  - Indice **di colonna.**
- In Java:
  - contengono **dati omogenei**.
  - Si possono estendere a dimensioni superiori (**array multidimensionali**).
- definizione: es. int[] [] biArray = new int[**NUM_COLONNE**][**NUM_RIGHE**]

Caratteristiche

- lunghezza
  - colonne: array.length
  - righe: array[Num_col].length

Algoritmi per Array

Generare Numeri Casuali

- Utilizzare metodo **statico** `Math.random()` per ottenere numeri **pseudo-casuali.**
  - restituisce un numero da [0,1[
  - numeri tra a e b:
    int n = (int)(a + (1 + b - a) * Math.random());
- classe **Random**
  - senza parametri
  - con parametri: seed = genera sempre la stessa sequenza
  - metodi: nextInt(), nextInt(bound) *bound = num massimo*

Trasformare Array in Stringhe

- il print(array) stampa il **riferimento**
- scorrere l'array con un **for** e salvare il valore di ogni cella **concatenandoli** uno ad uno in una **stringa**

Operazioni su Array

Eliminazione di Elementi

- **Senza mantenere l'ordine**:
  - **Sostituire** l'elemento da rimuovere con **l'ultimo** elemento.
  - **Ridimensionare** l'array/diminuire la dimensione logica
- **Mantenendo l'ordine**:
  - **Spostare** tutti gli elementi successivi di **una posizione indietro partendo dalla posizione successiva** a quella **rimossa.**
  - **Ridimensionare** o aggiornare la **dimensione logica.**

```
double[] values = {1, 2.3, 4.5, 5.6};
int indexToRemove = 1;

for (int i=indexToRemove; i<values.length-1; i++){
    values[i] = values[i + 1];
}

values = resize(values, values.length - 1);
```

Inserimento di Elementi

- **Senza mantenere l'ordine**:
  - ridimensionare/incrementare la dimensione logica
  - aggiungere l'elemento alla fine
- **Mantenendo l'ordine**:
  - ridimensionare/incrementare la dimensione logica**.**
  - Spostare tutti gli elementi successivi di una posizione avanti **partendo dall'ultimo**.
  - Inserire il nuovo elemento nella posizione desiderata.

-

```
double[] values = {1, 2.3, 4.5, 5.6};
values = resize(values, values.length + 1);
int index = 2;

for (int i = values.length - 1; i > index; i--){
    values[i] = values[i-1];
}
values[index] = 5.4;
```

Algoritmi di Ricerca

Ricerca Lineare

- Scansionare l'array dal primo all'ultimo elemento.
- Interrompere se il valore target è trovato.
-

```
for (int i = 0; i < vSize; i++){
    if (v[i] == target) {
        found = true;
        break;
    }
}
```

Ricerca del Minimo/Massimo

- **Assumere** che il **primo elemento** sia il minimo/massimo.
- Confrontare ogni elemento e **aggiornare il valore minimo/massimo** se necessario.

Ricerca di Posizioni Multiple

- Restituire un array contenente tutte le posizioni in cui si trova un valore target.

⛔ ECCEZIONI

Eccezioni

Gestione di Contenuti Condivisi in una Classe

Metodi Ausiliari

- Permettono di organizzare il codice **evitando duplicazioni**
- **sottoinsiemi** di istruzioni che possono essere **invocate da più metodi**
- Possono essere:
  - **Statici**: non legati a un oggetto
    - preceduto dal nome della classe dove è definito il metodo (es. Math.sqrt()).
    - solo nome del metodo se è definito nella stessa classe
  - **Non statici**: associati a un oggetto (es. .method()).
    - preceduto dal nome dell'oggetto (es persona.metodo()).
    - solo il metodo se fa riferimento all'oggetto della classe (sarebbe this.metodo() ma this è automatico e non serve)
- vantaggi:
  - Incrementano la **leggibilità**.
  - Favoriscono la **riusabilità del codice**.
- solitamente **privati**

Metodi Ausiliari - errori

- accedere da un metodo statico ad una variabile di esemplare

Lancio e gestione delle Eccezioni

Definizione

- Meccanismo per **gestire errori o situazioni inattese**.

Lancio delle eccezioni

- Quando un metodo **lancia un'eccezione**:
  - metodo **interrotto**
  - **chiamato il metodo** che invocava il metodo che ha lanciato l'eccezione → **propagata** l'eccezione
  - viene **interrotto** anche l'altro metodo
  - eccezione propagata fino al **main => arresto anormale del programma**

Gestione delle eccezioni

- Quando **NON è necessario terminare** il programma in caso di lancio di un'eccezione
- blocco **try-catch** per intercettare l'eccezione e gestirla
  - try contiene l'enunciato/gli enunciati che potrebbero lanciare l'eccezione
    - interrotto nel punto in cui si verifica l'eccezione
  - catch
    - tra parentesi tonde: tipo di eccezione
    - nel blocco: codice da eseguire se viene intercettata l'eccezione
  - più catch per differenziale la gestione in base all'eccezione intercettata

Diversi Tipi di Eccezioni

Tipologie Principali

La classe **Throwable** è la superclasse di tutte le eccezioni:

1. **Error**: problemi gravi a livello di sistema (non gestibili dall'utente).

    - gestione NON obbligatoria
2. **Exception**:
    - **eccezioni controllate**:
        - problemi che possono **verificarsi prima o poi**, indipendentemente dal programmatore
        - **Obbligatorio gestirle** con try/catch o dichiararle nel metodo (throws) altrimenti errore in compilazione
        - **Esempio**: IOException.
    - **RuntimeException** (eccezioni non controllate):
        - Non è obbligatorio gestirle.
        - **Esempi**:
            - ArithmeticException: errore matematico (es. divisione per zero).
            - NullPointerException: accesso a un riferimento nullo.
            - IndexOutOfBoundsException: indice non valido per array o stringa.

Argomenti Inattesi

Opzioni di Gestione

1. Non fare nulla (solo per metodi void).
2. Terminare il programma con System.exit(1).
3. **Lanciare un'eccezione** (es. IllegalArgumentException).
    a. enunciato **throw new**

Progettazione di eccezioni

Creazione di eccezioni personalizzate

- Si può definire una **nuova eccezione ereditando** da una **superclasse esistente** (es. RuntimeException).
    - **"vuote"**: ereditano tutti i comportamenti dalla superclasse
    - **nome personalizzato** (in evidenza)

- Le classi di eccezioni personalizzate servono a identificare **errori specifici.**

Tipi di eccezioni :

quale superclasse usare per la progettazione di eccezioni personalizzate?

- Sottoclassi di Error (legate a problemi della JVM, es. StackOverflowError).
  - gestite con try/catch.
  - non si devono definire sottoclassi di Error
  - riservate alla JVM
- Sottoclassi di Exception:
  - **Obbligatorie**: (IOException): Posso derivare da
    - Exception
  - **Non obbligatorie:** facoltative da gestire. Posso derivare da
    - RuntimeException

Lancio e gestione delle eccezioni

- **Parola chiave throw**:
  - Permette di lanciare oggetti di tipo Throwable o sue sottoclassi.
- **Clausola throws**:
  - Utilizzata nella **firma dei metodi** per dichiarare le eccezioni **obbligatorie** che possono essere lanciate.
  - obbliga l'uso di try-catch
- **Delegare la gestione**:
  - Un metodo con clausola throws può **"rilanciare" l'eccezione al metodo invocante** (deve avere anch'esso clausola throws)
- **Metodi utili nelle eccezioni**:
  - variabile **e:** di tipo "riferimento a una specifica eccezione"
  - getMessage(): Dettagli sull'errore.
  - printStackTrace(): Visualizza lo stack delle chiamate.

Utilizzo della Classe Scanner

Caratteristiche Principali

- Scanner può leggere input da:
    - Standard input (System.in).
    - Stringhe (new Scanner(String)).

Comportamenti Anomali

ATTENZIONE

```
do{
    try{
        n = console.nextInt();
        done = true;
    } catch(java.util.InputMismatchException e) {
            System.out.println("Riprova");
            console.next(); // o nextLine()
     }
} while (!done);
```

console.next(); è IMPORTANTISSIMO perché "pulisce" il buffer dalla parola che ha lanciato l'eccezione, altrimenti rimane lì e viene riletta

Scomporre Stringhe con Scanner

- Scanner può essere usato per analizzare una stringa come flusso di dati.
- creo l'oggetto Scanner con la **stringa come parametro esplicito**
- **delimitatori predefiniti:** spazi, tab, newline
- metodi
    - hasNext(): visualizza se ci sono parole successive a quella esaminata
    - next(): esamina la parola

Variabili Statiche

Definizione

- **Condivise** tra **tutti gli oggetti** di una classe. => esiste un'**unica copia**
- Dichiarate con il modificatore static.

Regole e Best Practices

- Devono essere **inizializzate** al momento della dichiarazione NON nel costruttore (*altrimenti il valore viene inizializzato ogni volta*)
- Devono essere **private** per evitare accessi indesiderati.
    - a volte sono pubbliche, ma final (es. Math.PI)

Progettazione di una Classe

Fasi

1. **Definizione delle caratteristiche essenziali**:
    - Elenco delle operazioni possibili (processo di astrazione).
2. **Creazione dell'interfaccia pubblica**:
    - Scrivere costruttori e metodi pubblici.
3. **Definizione delle variabili**:
    - Variabili di esemplare (private).
    - Eventuali variabili statiche (se necessario).

Chiamate ai metodi

Record di attivazione

- zona di memoria **riservata** nello Stack creata quando viene invocato un metodo
- contiene i suoi

    - parametri
    - variabili locali
    - punto di ritorno al codice che ha chiamato il metodo
- funzionamento:
    - Quando un metodo in esecuzione invoca un altro metodo viene **aggiunto in cima alla pila** il record di attivazione
    - Quando un metodo termina la sua esecuzione viene **tolto dalla pila** e si **torna ad eseguire il metodo** che l'aveva invocato

Tipi di Variabili

Ciclo di Vita

- **Variabili locali**: esistono solo all'interno del blocco dove si trova l'enunciato o esiste in tutto il programma
    - creata quanto viene eseguito l'enunciato in cui viene definita
- **variabili parametro**
    - creata all'invocazione del metodo ed eliminata quando il metodo termina
- **Variabili statiche**: esistono finché la classe è caricata
    - creata quando JVM crea la classe la prima volta ed eliminata quando viene scaricata .
- **Variabili di esemplare**: esistono finché l'oggetto è referenziato.
    - creata quando viene creato l'oggetto a cui appartiene ed eliminata quando l'oggetto viene eliminato

Inizializzazione

- **variabili di esemplare** e le **variabili statiche:** esplicitamente o automaticamente con valori di default
- **variabili parametro:** con valore dei parametri formali

- **locali:** non automaticamente → compilatore controlla che non vengano usate prima dell'assegnazione

Ambito di Visibilità

- Dipende dallo specificatore di accesso (public, private, ecc.).
- Variabili locali e parametro: visibili solo nel metodo.

Passaggio di Parametri

Parametri Formali ed Effettivi

- **Parametri Formali**: dichiarati nella firma del metodo.
- **Parametri Effettivi**: forniti nell'invocazione.
- Al momento dell'esecuzione:
    - I valori effettivi vengono copiati nei parametri formali.

Passaggio

- metodo invocante: interprete legge le variabili e genera il valore corrispondente
- metodo invocato: prima dell'esecuzione: automaticamente vengono definite le variabili parametro e inizializzate col valore

Passaggio per Valore

- avviene **sempre per valore**.
- **Cosa significa?**
    - Per i **tipi primitivi**, viene passata una copia del valore.
    - Per gli **oggetti**, viene passata una copia del riferimento all'oggetto.

attenzione: vai alla sezione ACHTUNG!!!

Passaggio di Tipi Primitivi

- Viene passata una **copia del valore.**

- Modifiche al parametro all'interno del metodo **non influenzano il valore originale.**
    - **necessario ritornare il valore modificato**

Passaggio di Oggetti

- Viene passata una copia del riferimento.
- Modificare le proprietà dell'oggetto all'interno del metodo **influenza l'oggetto originale**.

📂 GESTIONE FILE

Gestione dei file in Java

Reindirizzamento di Input e Output

Per **tenere traccia** dei dati inseriti/restituiti e per facilitare l'utente (*es. errore nell'inserimento dei dati)*

- **Input Standard**
  - Utilizzo del simbolo < per l**eggere** i dati da un file.
  - Comando: java Program < input.txt
- **Output Standard**
  - Utilizzo del simbolo > per **scrivere** i dati in un file.
  - Comando: java Program > output.txt
  - se il file non esiste viene **creato** altrimenti **sovrascritto**
- **Combinazione**
  - Input e output combinati: java Program < input.txt > output.txt

Pipes - canalizzazioni

- per **evitare** di utilizzare un **file temporaneo** per memorizzare l'output di un programma che serve da input ad un altro programma
- sintassi: java Programma1 < testo.txt | Java Programma2 > testoOrdinato.txt

Gestione dei File all'interno di un Programma

Utilizzando il pacchetto **java.io** della libreria standard

- **FileReader**
  - Per leggere un file di testo **esistente**
    - FileNotFoundException se file non esiste
    - obbligatoriamente gestita
  - Esempio: **FileReader** reader = new FileReader("file.txt");
  - costruttore ha bisogno del nome file
    - solo nome se si trova nella stessa cartella del programma
    - percorso se è in un'altra cartella
  - creazione di un esemplare **Scanner** che scansiona il file

        - Scanner sc = new **Scanner(**reader**)**
- **PrintWriter**
    - Per scrivere dati in un file di testo.
    - Esempio: PrintWriter writer = new PrintWriter("file.txt");
- **Chiusura dei file**
    - Importante chiudere file dopo l'uso per evitare instabilità.
    - può lanciare IOException => gestione obbligatoria

try-with-resources

- per una gestione più immediata delle eccezioni di tipo IOException

```
try(PrintWriter out = new PrintWriter(filename)){
    ...
    out.println("prova");
     ...
} // qui viene invocato out.close()
catch(...Exception e){
      // codice eseguito se e' stata lanciata l'eccezione del
 }finally{
      // codice eseguito in ogni caso
}
```

Formattazione numeri

Definizione e utilizzo

- scrivere i numeri NON nel formato standard ma (*per es.) con n cifre decimali e incolonnati*
- **printf:** contiene
    - **stringa di formato** con caratteri da stampare
    - **specificatori di formato:** comincia con **%**
    - **parametri:** calori da usare
    - *es. System.out.printf("Total:%5.2f", total); => total: 3.50 (5 caratteri (compreso punto e spazio) in totale di cui 2 dopo la vigola)*

RICORSIONE

Ricorsione

Definizione e Obiettivi

- **Ricorsione**: invocazione di un metodo all'interno di se stesso.

Esempi Classici

1. **Fattoriale**:
    - Definizione: **n!=n×(n-1)!**
    - **Caso base**: 0!=1
2. **Fibonacci**:
    - Definizione: **Fib(n)=Fib(n-1)+Fib(n-2)**
    - **caso base**
        i. Fib(0)=0
        ii. Fib(1)=1

Funzionamento

1. Metodo ricorsivo **chiama se stesso**
2. **JVM** sospende il metodo invocante
3. esegue il metodo → al suo termine **torna** dove era stato sospeso quello chiamante

Progettazione di Metodi Ricorsivi

- **Caso Base**
    - Risolve una **porzione semplice** del problema **senza invocazioni ricorsive.**
- **Passo Ricorsivo**
    - Risolve una parte **complessa ridotta** e **invoca se stesso.**
- **Composizione**
    - Combina i risultati per ottenere la soluzione completa.

Problemi nella Ricorsione

- **Ricorsione Infinita**
    - **Mancanza del caso base** o semplificazione inadeguata..
    - invocare ricorsivamente solo dopo aver **semplificato** il problema ossia **avvicinarsi al caso base**
    - Errore: StackOverflowError => lista di metodi in attesa si allunga e l'ambiente runtime **esaurisce la memoria**
- **Efficienza**
    - **Ricorsione multipla (**es. Fibonacci) è meno efficiente rispetto a soluzioni iterative.
        - quando il metodo chiama se stesso più di una volta

Eliminazione della Ricorsione

- **Ricorsione in Coda**
    - metodo esegue **una sola ricorsione** come **ultima azione**
    - Più semplice da eliminare, trasformandola in un **ciclo iterativo**.
- **Benefici**
    - La ricorsione in coda può rendere il **codice più leggibile**.

Ricorsione Strutturale

- Utilizzata per manipolare **strutture per la memorizzazione** come array e stringhe.
- **Esempio: Inversione di una Stringa**
    - Concatenazione del primo carattere con l'inversione della parte restante.

```
public static String reverse(String s){
    if(s == null){
        throw IllegalArgumentException;
    }
    if (s.length() < 2)
```

```
        return s; // caso base // invocazione con un problema più semplice
    }
    return s.charAt(s.length()-1) + reverse(s.substring(0,s.length()-1));
}
```

COMPLESSITA'

Complessità Computazionale

Obiettivi

- Comprendere i metodi di misura delle **prestazioni algoritmiche.**
- **Confrontare e classificare algoritmi** in base alla complessità.

Analisi delle Prestazioni

Tipi di Analisi

1. **Sperimentale**:
    - **Misura diretta dei tempi di esecuzione** su insiemi di dati.
    - Utilizzo di **System.currentTimeMillis()** in Java per calcolare la durata (in long).
        i. **differenza tra due risultati** di System.currentTimeMillis() (uno all'inizio e uno alla fine)
    - NON con cronometro
        i. si deve calcolare caricamento in memoria, classi, lettura dati
        ii. dovrei fare un calcolo medio (pk dipendono da altri programmi in corso sul computer)
    - **non è indipendente dalla macchina**
2. **Teorica**:
    - Basata su **tipo e frequenza** delle operazioni.
        i. operazioni primitive (*es. lettura, assegnazione valore a variabile, operazione aritmetica…)*
        ii. dimensione dati (**indicato con n)** e valore
    - Fornisce una stima qualitativa **indipendente dall'implementazione.**
    - comprende
        i. caso peggiore
        ii. caso medio

iii. **caso migliore**

Misure Principali

- **Tempo di Esecuzione**.
- **Memoria Occupata**.

Notazioni Asintotiche

O-Grande

$$f(n) \in O(g(n)) \quad se \quad \exists c > 0 \; e \; n_0 > 0 \quad tali\ che \quad f(n) \leq cg(n) \quad \forall n \geq n_0$$

- Rappresenta il **limite superiore** della complessità.
- Si considera solo il termine dominante della funzione.

Altre Notazioni

- **Omega-Grande**: limite inferiore.

  $$f(n) \in \Omega(g(n)) \quad se \quad \exists c > 0 \; e \; n_0 > 0 \quad tali\ che \quad f(n) \geq cg(n) \quad \forall n \geq n_0$$

- **Theta-Grande**: limite preciso (sia superiore che inferiore).

Complessità Algoritmica

Ottenere O-Grande

- costanti moltiplicative non vengono contate: *es 3 n => O(n)*

$$O(1) \subset O(logn) \subset O(n) \subset O(nlogn) \subset O(n^2) \subset O(n^3) \subset 2^n \subset n^n$$

- **due blocchi** B1 e B2
  - **sequenza o alternativi** (if) => complessità totale è il **massimo** tra le due complessità
  - **blocchi annidati =>** prodotto delle due complessità

Classificazione

1. **Costante (O(1))**:
   - Esempio: lettura di un valore da un array.
2. **Lineare (O(n))**:
   - Esempio: ricerca in una lista non ordinata.
3. **Quadratica (O(n²))**:
   - Esempio: cicli annidati.
4. **Logaritmica (O(log n))**:
   - Esempio: raddoppio dell'indice in un ciclo.

Classi di Complessità

- **P**: problemi risolvibili in tempo polinomiale.
- **NP**: problemi per cui non si conosce una soluzione polinomiale.

<u>DA RIVEDERE QUALCOSAAAAA !!!!</u>

Complessità Computazionale e Analisi di Complessità

Introduzione

- La **complessità computazionale** misura le risorse necessarie all'esecuzione di un algoritmo.
    - **Risorse**:
        - **complessità temporale: Tempo** di esecuzione. **T**(n)
        - **complessità spaziale: Spazio** occupato in memoria. **S**(n)
    - Misura assoluta: dipende dalla **dimensione dell'input n.**
    - **scopo:** calcolare **l'efficienza** di un algoritmo a prescindere da chi lo esegue

Complessità Temporale

Definizione e Calcolo

- T(n): misura il **tempo di esecuzione** in relazione alla dimensione dell'input.
- Non dipende dall'hardware o implementazione specifica, ma dal numero di operazioni fondamentali.

Calcolo

- individuando insiemi di **operazioni fondamentali** → tempo di esecuzione è l'unità di misura
    - **passo base**: tempo **costante** → non dipende dal tipo e dal valore dell'oggetto
        i. **assegnazione** di un valore ad una variabile
        ii. operazioni **aritmetiche**
        iii. **accesso** ad un elemento di un array
        iv. espressione booleana
    - altre non sono passi base

Complessità Spaziale

Definizione e Calcolo

- Espressa tramite la funzione **S(n),** misura la **memoria occupata.**
  - **Costante**: inizializzazione di variabili.
  - **Variabile**: allocazione di array o strutture ricorsive.

Esempi Pratici

1. **Fattoriale Ricorsivo**:
   - Complessità spaziale O(n) dovuta alla pila di chiamate ricorsive.
2. **Fattoriale Iterativo**:
   - Complessità spaziale O(1) grazie all'uso di variabili temporanee.
3. **Merge Sort Ricorsivo**:
   - Complessità O(nlogn) per la gestione della ricorsione.

Analisi Asintotica e Classi di Complessità

Notazione O-Grande

- Classifica algoritmi in base alla crescita della funzione complessità.
  - **Esempi**:
    - O(1): Determinare se un numero è pari.
    - O(logn): Ricerca binaria.
    - O(n): Ricerca del massimo in un array.
    - O(n^2):: Ordinamento tramite Bubble Sort.
    - O(n!): Generazione di tutte le permutazioni.

Progettazione e Ottimizzazione

- La **complessità di un problema** è intrinseca e deve essere studiata **prima** di sviluppare un algoritmo.
  - Valutare la **dimensione dell'input** e i vincoli temporali/spaziali.

- Esempi di approccio:
    - **Scomposizione** del problema in sottoproblemi.
    - Trovare un equilibrio tra **efficienza e semplicità.**

🥇 ORDINAMENTO DI DATI

Ordinamento dei Dati

Motivazioni

- **Obiettivi principali**:
  - Ordinare dati (numeri, stringhe, conti) secondo un criterio..
  - una sequenza di **dati confrontabili** si dice **ordinata** quando $a_i \leq a_{i+1} \forall i = 0, \quad n - 2$
- **Utilizzi pratici**:
  - Ricerca efficiente (es. dizionari).
  - Trovare min/max o k-esimo min/max.
  - Verificare unicità o identificare duplicati.

Tipologie

- **sul posto:** metodo di ordinamento riceve il contenitore e ne ordina il contenuto **senza ritornare nulla**
- **non sul posto:** metodo riceve il contenitore e **restituisce** un altro contenitore con **gli stessi elementi ordinati**

sono **statici** e su contenitori "pieni" o riempiti in parte se si aggiunge la dimensione logica come parametro

Selection Sort

---

1. **Descrizione**:
   - Trova la **posizione dell'elemento minimo** in una porzione non ordinata.
   - Lo **scambia** con l'elemento alla **posizione corrente.**
   - **Ripete** per tutte le successive posizioni fino alla fine (parte da ordinare contiene un elemento).

```
public static void selectionSort(double[] a){
    if (a == null)
        return;

    for (int i = 0; i < a.length - 1; i++) {
        int minPos =i;
        for(int j = minPos+1;  j < a.length; j++){
            if(a[j] < a[minPos ]){
                minPos = j;
            }
        }
        if (minPos != i) {
            double temp = a[i];
            a[i]  = a[minPos];
            a[minPos] = temp;
        }
    }
}
```

2. **Vantaggi e Svantaggi**:
    - Semplice da implementare, ma inefficiente per grandi dataset.
3. **Complessità:**
    - **Caso peggiore/medio**: $O(n^2)$
    - **Caso migliore:** $O(n^2)$
        i. Non trae vantaggio da un array già ordinato.
    - Complessità invariata tra caso migliore, medio e peggiore.
4. **adattamenti:**
    - **stringhe:** compareTo() per confrontare
    - **ordinamento decrescente:** cambiare con >
    - **altri tipi primitivi:** cambio il tipo dell'array
    - **array riempiti in parte:** size (dimensione logica) invece che dimensione fisica

MergeSort

---

1. **Descrizione**:

- Divide l'array in **due metà.**
- **Ordina ricorsivamente** entrambe le metà.
- **Combina** le due metà ordinate in un'unica sequenza.

```
public static void mergeSort(double[] a){
    if (a == null)
        return;
   if(a.length <2){
        return;

    int mid = a.length/2;
    double[] right = new double[mid];
    double[] left = new double[a.length - mid];

    System.arraycopy(a, 0, left, 0, left.length);
   System.arraycopy(a, mid, right, 0, right.length);

    mergeSort(left);
   mergeSort(right);

    merge(a, left, right);
 }

public static void merge(double[] a, double[] left, double right){
    int ia = 0, ileft = 0, iright =0;
  while(ileft < left.length && iright < right.length){
        if(left[ileft] < right[iright]{
              a[ia++] = left[ileft++];
        }else{
              a[ia++] = right[iright++];
        }
     }
  while(ileft < left.length){
          a[ia++] = left[ileft++];
   }
    while( iright <  right.length){
          a[ia++] =  right[ iright++];
   }
 }
```

2. **Caratteristiche**:

- **Prestazioni migliori** rispetto al Selection Sort.
- Complessità per tutti i casi **O(n log n)**
- **Non opera "sul posto"**: richiede spazio aggiuntivo.

InsertionSort

- **Funzionamento:**
  - Considera il **primo elemento come ordinato** (elemento unitario).
  - Inserisce il prossimo elemento nell'analisi
    - **confronta con la parte ordinata** e lo **sposta a sinistra** di n posizioni necessarie
  - **Ripeti** fino alla fine.
  -

```
public static void insertionSort(double[] a){
    for(int i = 1; i <a.length ; i++){
        for(int j = i-1; j >= 0 ; j–){
            if(a[j] > a[j+1]){
                double temp = a[j];
                a[j]  = a[j+1];
                a[j+1]  = temp;
            }else{
                break;
            }
        }
    }
}
```

- **Complessità:**
  - Caso migliore: O(n).
  - Caso medio/peggiore: $O(n^2)$
- **Utilizzo:** Efficace per piccoli array o array quasi ordinati.
  - **inserire** un nuovo elemento su un array **già ordinato** (pk si trova nel caso migliore → **O(n)**)
    - eseguo solo il ciclo interno partendo da j = a.length -2

Confronto Tra Algoritmi

Algoritmo	Caso migliore	Caso medio	Caso peggiore
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
MergeSort	O(n logn)	**O(n logn)**	**O(n logn)**
Insertion Sort	**O(n)**	$O(n^2)$	$O(n^2)$

Algoritmi di ricerca

Ricerca Sequenziale

- **Descrizione:** Scansione elemento per elemento.
- **Complessità:**
  - Caso migliore: **O(1)**
  - Caso medio/peggiore: **O(n)**

```
public static int sequentialSearch(int[] a, int v)[
   if (a == null){
        return -1; // NON TROVATO
    }
    for (int i = 0; i < a.length; i++){
        if (a[i] == v){
           return i; // TROVATO
        }
    }
    return -1; // NON TROVATO
}
```

Ricerca Binaria

- **Requisito:** Array ordinato.
- **caratteristiche**: **ricorsivo**
- **Funzionamento:**
    - Confronta il valore cercato con **l'elemento centrale.**
    - cerca ricorsivamente nella **metà di destra** se è **maggiore** dell'elemento centrale analizzato, altrimenti a **sinistra**
- **Complessità:** O(logn)

```
public static int binarySearch(int[] a, int target)[
      return binSearch( a, 0, a.length -1; target);
}

public static int binSearch(int[] a, int  from ,int to , int  target){

   if (to < from){
        return -1; // NON TROVATO
    }
    int mid = (to + from)/2;
    int middle = a[mid];

    if (middle == target){
            return mid; // TROVATO
    }else if(middle > target){
            return binSearch( a, from, mid-1; target);
   }else{
            return binSearch( a, mid+1, to; target);
   }
}
```

Ordinamento e ricerca di oggetti

Algoritmi di ordinamento

- Gli algoritmi usati per ordinare numeri possono essere adattati per oggetti, purché questi siano **confrontabili**.
    - Interfaccia Comparable (*vedi sezione Interfacce*)
    - criterio dipende dalla **classe** che definisce l'oggetto
- **Metodo compareTo()**:
    - Determina l'ordine tra oggetti:
        - Ritorna un intero **negativo** se l'oggetto corrente **precede** quello confrontato.
        - Ritorna **zero** se sono **uguali**.
        - Ritorna un intero **positivo** se l'oggetto corrente **segue** quello confrontato.

CompareTo per gli oggetti

- **Classe String**:
    - Implementa il metodo compareTo() per l'ordinamento lessicografico.
- **Classe personalizzata (BankAccount)**:
    - Deve implementare compareTo() per definire l'ordine
        - secondo alcune variabili d'istanza
    - **riscrivere i metodi** di ordinamento: devono ricevere un array di oggetti

🧓 EREDITARIETA'

Ereditarietà

Cos'è l'Ereditarietà

- Strumento dell'OOP per:
  - **Riutilizzare** codice esistente
  - Creare **classi più specializzate** da classi generali.
    - efficiente: se cambio la classe generale non devo cambiare anche quella specializzata
- **Esempio:** SavingsAccount (conto di risparmio) estende BankAccount (conto bancario).
- **Sintassi**: public class NomeSottoclasse **extends** NomeSuperclasse { ... }
- anche a **più livelli**

ogni classe ne estende un'altra e SOLTANTO una

Superclasse e Sottoclasse

- **Superclasse:** Classe generica da cui si ereditano caratteristiche.
  - **superclasse universale: Object** => ogni classe deriva implicitamente da essa
- **Sottoclasse:** Classe più specifica che aggiunge o sovrascrive funzionalità
  - eredita tutte le caratteristiche (metodi e variabili)
  - esemplari si usano come se fossero esemplari della superclasse con funzionalità in più

Metodi e Variabili nelle Sottoclassi

Metodi delle Sottoclassi

- Tre possibilità:
  1. **Nuovi metodi:** Aggiunti nella sottoclasse.

2. **Eredità:** Metodi della superclasse riutilizzati.
3. **Override:** metodi della superclasse vengono **sovrascritti**.

Sovrascrittura dei Metodi

- **Modifica il comportamento** di un metodo della superclasse nella sottoclasse.
- scrivere un metodo con la **stessa firma** del metodo della superclasse
- **prevale** su quello della superclasse su oggetti della sottoclasse
- **Uso di super:** Per invocare il metodo originale della superclasse.
  super.metodo();

Costruttori

Uso di super nei Costruttori

- **super**: Invoca un costruttore della superclasse.
  - Deve essere il **primo comando** nel costruttore.
  - in realtà la **JVM invoca sempre super** senza parametri implicitamente
  - può avere dei **parametri**

Costruttore Predefinito

- costruttore senza parametri
- Esiste **implicitamente solo se** la classe non definisce altri costruttori.
- **Errori comuni:**
  - se è presente un costruttore con parametri nella superclasse e si invoca super() (senza parametri, costruttore predefinito) nella sottoclasse → errore del compilatore
  - se nella superclasse esiste un costruttore esplicito => deve esistere almeno un costruttore esplicito nella sottoclasse

Conversioni tra Superclasse e Sottoclasse

Up-Casting

- Assegna un **riferimento di una sottoclasse** a una variabile di tipo **superclasse**.
  - *es.* **BankAccount** bAccount = new **SavingsAccount**(10);
  - **Perdita di accesso ai metodi specifici** della sottoclasse.
- **scopo:** creare delle funzionalità che operano su oggetti di tipo superclasse → posso utilizzare esemplari derivati da superclasse con up-casting
  - altrimenti dovrei creare un metodo per ogni tipo di classe derivata
- **avviene anche implicitamente**
- il riferimento della sottoclasse non convertito funziona ancora con tutte le sue potenzialità

Down-Casting

- Convertire un riferimento di tipo superclasse in uno di tipo sottoclasse.
- Necessario verificare il tipo con instanceof per evitare errori.

figli

- classe C deriva da B
- B deriva da A

C non può accedere ai metodi della classe A

- come stabilire di che tipo è l'istanza? operatore **instanceof**
  - **es.** variabileOggetto **instanceof** NomeClasse
  - restituisce boolean

Polimorfismo

Definizione

- Un oggetto di una sottoclasse può essere usato come se fosse della superclasse.
- **L'invocazione dei metodi è determinata dal tipo dell'oggetto, non dal tipo della variabile**
    - es. **BankAccount** bAccount = new **SavingsAccount**(10); => vengono invocati i metodi di SavingsAccount se sovrascritti
    - **Early Binding:** Scelta del metodo in fase di **compilazione** (es. overload - metodi sovraccarichi - nome metodo uguale ma parametri diversi).
    - **Late Binding:** Scelta del metodo in fase di **esecuzione** (es. override - stessa firma).

up-casting - polimorfismo (incongruenze)

Controllo di accesso

- Modificatori: public, private, protected, package.
- **package:**
    - accesso **di default**
    - accessibile da metodi di classi **nello stesso package**
    - **errore**: lasciare le variabili di esemplare senza specificatore
- **Protected:**
    - per variabili e metodi
    - accessibili da tutte le **sottoclassi**
    - **parziale violazione dell'incapsulamento**

Modificatore final

- **Metodi:** Impedisce l'override.
- **Classi:** Impedisce la creazione di sottoclassi.

superclasse universale Object

- ogni classe di Java eredita **tutti i metodi** di Object
    - **toString()**
    - **equals(Object other)**
    - **clone()**

Sovrascrivere toString ed equals

- **toString:** Per una rappresentazione testuale utile dell'oggetto.
    - perché la sua invocazione restituisce la **descrizione testuale standard** (nome della classe@ numero indirizzo in memoria)
    - deve produrre una **stringa** contenente le info di stato dell'oggetto
        - valore delle variabili di esemplare
        - valore di variabili statiche
- **equals:** Confronto tra oggetti basato sulle variabili di stato.
    - perché restituisce **true** solo se gli oggetti coincidono
    - Necessario fare **casting esplicito** al tipo corretto.

```
public boolean equals(Object otherObject){
    BankAccount otherAccount = (BankAccount)otherObject;
    return balance == otherAccount.balance;
}
```

---

7. Errori Comuni

- **Ombreggiamento delle Variabili:**
    - Definire una variabile con **lo stesso nome** nella sottoclasse, causando incoerenze.
    - accesso privato a una variabile della superclasse

- **Costruttori Incompatibili:** Dimenticare di invocare costruttori della superclasse.

🙂 INTERFACCE

Interfacce

Concetti fondamentali

- Un'interfaccia definisce **comportamenti astratti** che devono essere **implementati dalle classi** che dichiare di avere quel comportamento
- Le classi possono implementare **più interfacce contemporaneamente.**
- definiscono solo **la firma dei metodi NON statici**
    - implicitamente **public**
- **NON** possono avere:
    - Costruttori.
    - Variabili di istanza.
    - Metodi con implementazione.
- Le interfacce **astraggono comportamenti comuni,** favorendo il **riuso** del codice.

Sintassi

- sintassi: public **interface** NomeInterfaccia

Classi che implementano un'Interfaccia

- sintassi: public class NomeClasse **implements** NomeInterfaccia
- Deve implementare **tutti** i metodi specificati nell'interfaccia → stessa **firma**

Interfaccia Comparable

- **ordine di oggetti:** se sono confrontabili tramite interfacce come Comparable.
- oggetti confrontabili: algoritmi di ordinamento non hanno bisogno di conoscere i particolari dell'oggetto
- definita in **java.lang**

compareTo()

- definito nell'interfaccia Comparable

```
private static void swap(Comparable[] a, int i, int j){
    Comparable temp = a[i];
    a[i] = a[j];
    a[j] = temp;
}

private static int findMinPos(Comparable[] a, int from){
    int pos = from;
    for (int i = from + 1; i < a.length; i++){
        if (a[i].compareTo(a[pos]) < 0) {
            pos = i;
        }
    }
    return pos;
}
```

Conversione da classe a interfaccia

- Una variabile **dichiarata con il tipo di un'interfaccia** può riferirsi a un oggetto di una classe che implementa tale interfaccia.
    - Esempio: Comparable c = new BankAccount(10);
        - può usare compareTo
- **Limiti**: Non è possibile **utilizzare metodi** della classe tramite il riferimento all'interfaccia.

Conversione da interfaccia a classe

- Si può eseguire un **cast esplicito** per ottenere un riferimento al **tipo di classe concreto** dall'interfaccia.
- **Rischio**:
  - Se il **cast è errato** (il tipo dell'oggetto non corrisponde alla classe), viene lanciata una ClassCastException durante l'esecuzione.
  - controllo con **instanceof**

Uso della conversione

- **Da classe a interfaccia**:
  - Adatta l'oggetto per essere utilizzato in algoritmi generici basati sull'interfaccia.
    - es. ordinamento

// Nicolò: questo è il punto che guardavamo venerdi su Set s = new ArraySet()

- **Da interfaccia a classe**:
  - Permette di accedere ai metodi specifici della classe concreta.

Con queste conversioni, si combina l'astrazione dell'interfaccia con la flessibilità della classe concreta, mantenendo il codice leggibile e riutilizzabile.

ADT

ADT

Paradigmi OOP e Strutture Dati

Principi dell'OOP

- **Robustezza**:
  - Programmi capaci a gestire **situazioni inaspettate** (input inattesi)
- **Adattabilità**:
  - Capacità di **evolvere** e funzionare su **architetture diverse**.
- **Riusabilità**:
  - Codice modulare, facile da integrare in contesti diversi.

Concetti OOP

- **Astrazione**:
  - Definire i concetti essenziali di un sistema.
- **Incapsulamento (Information Hiding)**:
  - **Nascondere i dettagli intern**i e fornire solo l'interfaccia esterna.
- **Modularità**:
  - Suddivisione in componenti funzionali separati.
- **Caratteristiche principali**:
  - Classi e oggetti.
  - **Ereditarietà** per specializzare il comportamento.
  - **Polimorfismo** per trattare oggetti di classi diverse uniformemente.
    - Il tipo di una variabile non determina completamente il tipo dell'oggetto a cui essa si riferisce

Strutture dati e tipi di dati astratti

Definizione: Contenitori che organizzano i dati in un formato specifico

- per esigenze di **tempo** di costruzione e accesso ai dati
- occupazione di **memoria**

Strutture dati

- **Classificazione**:
  - **Lineari**: array, liste concatenate.
  - **Non lineari**: alberi, grafi.
- **definizione**: modo sistematico per **organizzare i dati** in un contenitore controllando le **modalità di accesso**

Tipi di dati astratti (ADT)

- Rappresentano **modelli di strutture dati:**
  - Specificano il tipo di dati e le operazioni supportate da una struttura dati
    - struttura definisce **concretamente come** vengono eseguite le operazioni
  - Realizzati in Java tramite **interfacce**.
  - mette a disposizione
    - costruttore
    - **metodi** per: inserimento, rimozione, ricerca

Container

- contenitore generico
- le ADT **estendono** Container
- metodi
  - isEmpty
  - makeEmpty

Pila

- **LIFO** (Last In, First Out).
  - Unico dato ispezionabile e rimovibile è **in cima**
- Operazioni: **push**, **pop**, **top**.

- Applicazioni: undo in editor, cronologia nei browser, metodi JVM

QUEUE

- **FIFO** (First In, First Out).
    - Unico dato ispezionabile e rimovibile è **in posizione iniziale**
- Operazioni: **enqueue, dequeue, getFront.**
- Utilizzi: gestione delle richieste in coda.

Deque (Coda doppia):

- Inserimenti e rimozioni possibili da **entrambe le estremità.**
- estende **Container** NON Queue

Strutture dati associative

---

organizzazione dei dati secondo **relazioni binarie**

- chiave
- valore

Mappe

- Associa **chiavi uniche** a valori.
    - a ciascuna chiave corrisponde **un solo** valore
- Supporta operazioni
    - **inserimento**: coppia chiave valore
    - **rimozione e ricerca**: attraverso la chiave
    - **keys[]**: restituisce array di chiavi
- possibile creare una Mappa con coppie ordinate per chiave
    - aggiungo metodo: Comparable[] sortedKeys();
        - restituisce un array di chiavi ordinate
    - creo interfaccia che estende **Map**
    - Aggiunta di valori con chiave **comparable**
        - valore inserito secondo l'ordine

Dizionari

- Simile alla **mappa**, ma permette **chiavi duplicate.**
    - Ad esempio, traduzioni di una parola con significati multipli.
    - chiavi **associabili a più valori** → appaiono più volte
        - posso usare un array per contenere i valori
        - più coppie con stessa chiave
- **ordinata**
    - le chiavi appartengono a una classe che implementa **comparable**
    - **compareTo()** per cercare e ordinare
- **non ordinata**
    - **equals():** per ordinare e cercare

Tabelle (Table)

- Mappa con **chiavi numeriche** in un **intervallo** noto a priori
    - chiave = indice dell'array
    - valori con key non presente: **null**
- Prestazioni **O(1)**,
    - inefficiente in termini di **memoria** se le chiavi sono disperse.
        - occupazione di n valori non dipende da n
    - **fattore di riempimento:** numero dati contenuti diviso per la dimensione massima

Hash Table

- intervallo delle chiavi è **noto a priori**
- Risolve il problema delle chiavi sparse con una **funzione di hash**.
- Utilizza liste (bucket) per gestire collisioni.
- Prestazioni dipendono dalla funzione di hash e dal numero di bucket.

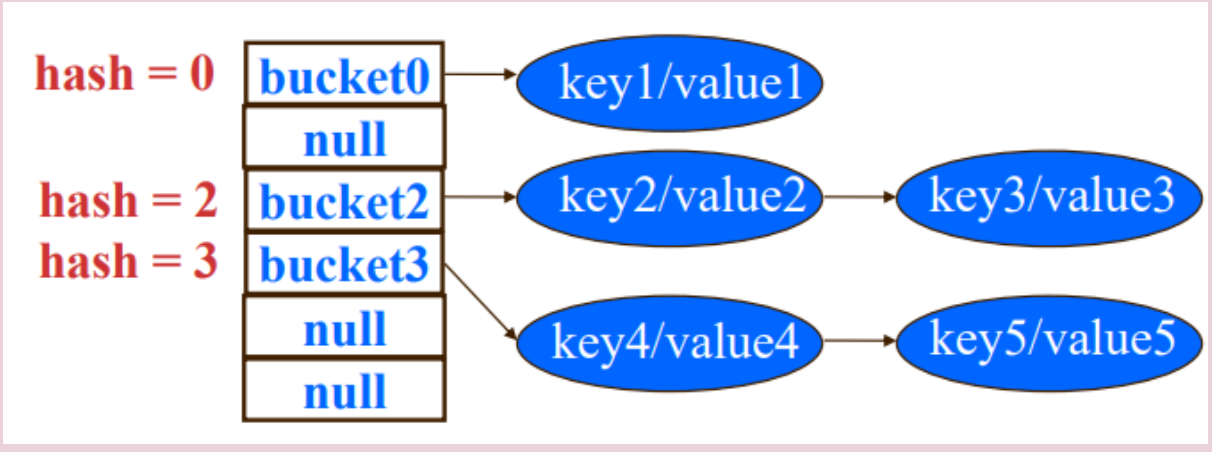
Funzione di Hash

- **dominio:** insieme di chiavi che identificano **univocamente** i dati

- **codominio:** insieme degli indici validi per accedere agli elementi → **chiave ridotta**
- es. Kr = k % dim
- chiavi ridotta ha più valori associati => hanno chiave (normale) diversa

Collisioni

- Se due elementi hanno pari chiave ridotta
  - secondo le regole della tabella, il nuovo sovrascrive il vecchio
- soluzione: ogni valore viene salvato con la chiave vera
  - **lista** per ogni coppia **chiave-valore** con **pari chiave ridotta**
  - **tabella di hash con bucket**
    - **caso peggiore O(n):** tutti i dati in un'unica lista
    - **caso migliore O(n/M):** chiavi ridotte distribuite uniformemente, ogni chiave ridotta ha pari numero di elementi nella lista
      - n/M = dimensione lista
      - M = dimensione tabella
      - O(1) se tabella ridimensionata con M = n



Insiemi (Set)

- **Caratteristiche**:
  - Contenitore di **oggetti univoci** senza ordine specifico.
  - non tiene conto dell'ordine di inserimento dei dati

- **metodi:**
    - void add(Object obj)
    - boolean contains(Object obj)
    - Object[] toArray()
    - **non esiste rimozione**
- **Operazioni principali**:
        - ritornano un nuovo set con gli elementi con le proprietà
    - **Unione**: elementi presenti in almeno uno dei due insiemi.
    - **Intersezione**: elementi comuni a entrambi.
    - **Sottrazione**: elementi di un insieme che non appartengono all'altro.

PILA

Realizzazione della pila

Struttura base

- Uso di un **array "parzialmente riempito"** per rappresentare la pila.
- Problemi da affrontare:
  - **Array pieno (metodo push)**:
    - Lanciare un'eccezione (**FullStackException**).
    - **Ridimensionare** l'array.
  - **Pila vuota (metodi pop/top)**:
    - Lanciare un'eccezione (**EmptyStackException**).

Eccezioni nella pila

- Classi per eccezioni:
  - EmptyStackException e FullStackException estendono **RuntimeException =>** programmatore non è obbligato a gestirle

Caratteristiche

- gestisce qualsiasi tipo di dato
  - riferimenti ad **oggetti**
  - tipi **primitivi** con **auto-boxing**

Auto-boxing

Conversione automatica dal tipo primitivo a una variabile della corrispondente classe involucro o **Object**
es. **Integer** intObj = 2
Double doubObj = 2 NON funziona → con 2.0 si

Implementazione senza ridimensionamento

Metodi principali

- **push(Object obj)**: Inserisce un elemento, lancia FullStackException se l'array è **pieno**.
- **pop()**: Estrae **l'elemento in cima**, lancia EmptyStackException se la pila è **vuota**.
- **top()**: **Restituisce** l'elemento in **cima** senza rimuoverlo.

Implementazione con ridimensionamento

- **extends ArrayStack**
- **Sovrascrivere push**:
  - **Ridimensionamento automatic**o dell'array (resize) quando pieno.
- Uso di variabili protette (protected) per accedere ai membri della superclasse.

Prestazioni della pila

FixedArrayStack

- Tempo **costante O(1)** per tutte le operazioni.

GrowingArrayStack

- Tempo O(n) per operazioni di **ridimensionamento** (nel caso peggiore).
- **Analisi ammortizzata**:
    - **n-1** operazioni richiedono O(1) e **una** richiede O(n) => Costo medio di ogni operazione: **O(1).**
    - Se però, la ridimensione: n elementi diventano n+k => **O(n)**

Pila "sicura"

Problema

- Introduzione accidentale di **oggetti disomogenei** in una pila di oggetti omogenei.
- ATTENZIONE: il risultato **non dipende dal tipo dichiarato** per la variabileOggetto, ma dal tipo dell'oggetto a cui la variabile si riferisce

Soluzioni

- Controllo del tipo nel metodo push:
    - Uso di **instanceof** per verificare il tipo.

```
public class StringArrayStack implements Stack{
    public void push(Object obj){
        if(!(obj instanceof String)){
            throw new InvalidInputExceptio();
        }
        if (vSize == v.length)
            v = resize(v, 2*v.length);
        v[vSize++] = obj;
    }
}
```

    - Eccezione personalizzata: **InvalidTypeException**.
- **Ereditarietà**:
    - Sovrascrivere push nella sottoclasse (es. StringArrayStack).

- Evita duplicazioni di codice.

🐈‍⬛ CODA

Coda (Queue)

Definizione e Operazioni

- **Operazioni principali**:
  - **enqueue**: Inserisce un elemento.
  - **dequeue**: Rimuove il primo elemento.
  - **getFront**: Restituisce il primo elemento senza rimuoverlo.
- Realizzata con un array "**riempito solo in parte":**
  - Inserimento a destra.
  - Rimozione a sinistra.

Eccezioni

- **Errori comuni**:
  - **EmptyQueueException**: Accesso a una coda vuota.
  - **FullQueueException**: Coda piena.

Implementazioni della coda

Coda con due indici (FixedArrayQueue)

- Introduce due indici:
  - **front**: Indica il primo elemento.
  - **back**: Indica il primo spazio libero.
  - numero elementi: **back - front**
- Operazioni in tempo **O(1)**
- **problema:** inefficienza dello spazio
  - aggiungo **n** elementi => tolgo **n** elementi. La coda è **vuota** ma alla prossima aggiunta → **FullQueueException** perché back si è spostato fino al termine

Array circolare

- Gli indici **ritornano all'inizio** quando raggiungono la fine.
  - metodo **increment**
- Utilizza **n-1 elementi** per distinguere tra coda piena e vuota.
- Prestazioni: Tutte le operazioni in **O(1).**

Coda circolare ridimensionabile

- Ridimensionamento dell'array in caso di overflow:
  - Richiede **O(n)** per il ridimensionamento.

	slowFixedArray	FixedArray	GrowingArray	FixedCircArray	GrowCircArray
enqueue	O(1)	O(1)	O(1)*	O(1)	O(1)*
getFront	O(1)	O(1)	O(1)	O(1)	O(1)
dequeue	O(n)	O(1)	O(1)	O(1)	O(1)

÷ SET

Insieme (Set)

Definizione e Operazioni

- Contenitore di oggetti **distinti - NON duplicati**:
- non ha ordinamento
- **non ricorda l'ordine di inserimento**
    - **Add**: Inserisce un elemento.
    - **Contains**: Verifica la presenza di un elemento.
    - **toArray**: Restituisce un array degli elementi.

Operazioni

- **Unione**: elementi presenti in **almeno uno** dei due insiemi.
- **Intersezione**: elementi comuni a entrambi.
- **Sottrazione**: elementi di un insieme che non appartengono all'altro.

Implementazioni

- **Array non ordinato (ArraySet)**:
    - Operazioni **O(n)**: add utilizza contains.
    - Operazioni tra insiemi (unione, intersezione, sottrazione) **$O(n^2)$.**
- **Array ordinato (ArraySortedSet)**:
    - Inserimenti in **O(n)** con **insertion sort**.
    - contains: **Ricerca binaria** in **O(logn)**
    - Operazioni tra insiemi ottimizzate a **O(n logn)**

TABELLA

Tabella (Table)

Definizione

- Mappa con **chiavi numeriche intere.**
- Realizzata con un array:
  - Le chiavi fungono da **indici**.
- celle vuote → valore **null**

Prestazioni

- **Operazioni in O(1)**:
  - **insert**: Inserisce un elemento.
  - **remove**: Rimuove un elemento.
  - **find**: Trova un elemento.

Limiti

- **Chiavi**:
  - Devono essere intere (eventualmente normalizzate).
- **Memoria**:
  - Spreco di memoria con fattori di riempimento bassi.
  - Problemi con chiavi sparse.

Array ridimensionabile

- Richiede **O(n)** per il ridimensionamento:
  - Peggiora le prestazioni nel caso peggiore.

MAPPA

Mappa (Map)

Definizione e Operazioni

- **ADT Map**:
  - Contenitore di **associazioni chiave/valore.**
  - Operazioni principali:
    - **put(key, value):** Inserisce o sostituisce un'associazione e se gia' presente restituisce il valore associato alla chiave appena sostituita.
    - **get(key):** Restituisce il valore associato a una chiave.
    - **remove(key):** Rimuove l'associazione e restituisce il valore.
    - **keys():** Restituisce un array con le chiavi.

Implementazione con Array riempito in parte

- Ogni cella contiene una coppia chiave/valore (**Pair**).
- Operazioni:
  - **Inserimento**:
    - **O(n)**: Ricerca per chiave prima dell'inserimento.
    - Senza vincolo di chiave unica: **O(1)**
  - **Rimozione**:
    - **O(n):** Ricerca e spostamento dell'ultimo elemento.
  - **Ricerca**:
    - **O(n)**: **Ricerca sequenziale.**

Classi interne

Definizione

- Una classe può essere definita all'**interno di un'altra classe** per descrivere un tipo logicamente correlato.
- Caratteristiche:
    - Le classi interne condividono una "relazione di fiducia" con la classe esterna:
        - Accesso reciproco a **metodi** e **campi**, anche se privati.
    - Un oggetto di una classe interna è **sempre associato a un oggetto della classe esterna.**

Vantaggi

- Incapsulamento più stretto:
    - La classe interna può essere resa privata e non accessibile dall'esterno.
- Maggiore modularità:
    - Rende il codice più leggibile e logico.

Limitazioni

- Un oggetto di classe interna può essere creato solo:
    - **Dentro metodi non statici** della classe esterna.
    - Utilizzando la sintassi qualificata ClasseEsterna.ClasseInterna.
-

```
public class ClassiInterneTester{
    public static void main(String[] args){
        ClasseEsterna e = new ClasseEsterna(); //tutto ok

        //MAI LECITO: non si possono creare ogg. di ClInterna qui
        ClasseEsterna.ClasseInterna obj = new ClasseEsterna.ClasseInterna();

        //MAI LECITO: il tipo deve essere "ClasseEsterna.ClasseInterna"
        ClasseInterna i = e.metEsterno();

        //LECITO SOLO SE ClasseInterna e` public in ClasseEsterna
        ClasseEsterna.ClasseInterna i = e.metEsterno();
```

```
            //LECITO SOLO SE sia ClasseInterna che il metodo sono public
            (e.metEsterno()).metInterno();

             //LECITO SOLO SE sia ClasseInterna che il campo sono public
            (e.metEsterno()).campointerno = 1;

            //SEMPRE LECITO (ma inutile se ClasseInterna e` private)
            e.metEsterno();
        }
}
```

Compilazione

- Il compilatore genera **due file .class distinti:**
  - **ClasseEsterna.class**: Rappresenta la classe esterna.
  - **ClasseEsterna$ClasseInterna.class**: Rappresenta la classe interna.

Mappa Ordinata (SortedMap)

Definizione

- Chiavi devono essere **Comparable**.
- Aggiunge il metodo **sortedKeys():**
  - Restituisce le chiavi ordinate.

Implementazione con Array Ordinato

- Le coppie **chiave/valore sono ordinate.**
- Operazioni:
  - **Inserimento**:

        - **O(n)**: Ordinamento per **inserzione. Eccezione** se si prova ad aggiungere un oggett**o non comparable**
    - **Rimozione**:
        - **O(n): Spostamento** degli elementi per mantenere l'ordine.
    - **Ricerca**:
        - **O(logn)**: **Ricerca binaria.**

SortedMap	In array ordinato	In array non ordinato
ricerca	O(log n)	O(n)
Inserimento	O(n)	O(n)
rimozione	O(n)	O(n)

📖 DIZIONARIO

Dizionario (Multimap)

Definizione e Differenze

- Contenitore chiave/valore come Map, ma:
  - Le chiavi **non devono essere uniche**.
  - Una chiave può essere associata a più valori.

Operazioni

- **Insert**: Aggiunge un'associazione.
- **Remove**:
  - Rimuove associazioni per una chiave.
  - Restituisce i valori associati come array.
- **Find**:
  - Restituisce uno o tutti i valori associati a una chiave.
- **Keys**: Restituisce un array con le chiavi (potrebbero essere ripetute).

Implementazione con Array

- **Array Ordinato**:
  - **Ricerca**: O(logn)(ricerca binaria).
  - **Inserimento/Rimozione**: O(n)
- **Array Non Ordinato**:
  - **Ricerca**: O(n) ricerca lineare.
  - **Inserimento**: O(1).
  - **Rimozione**: O(n)

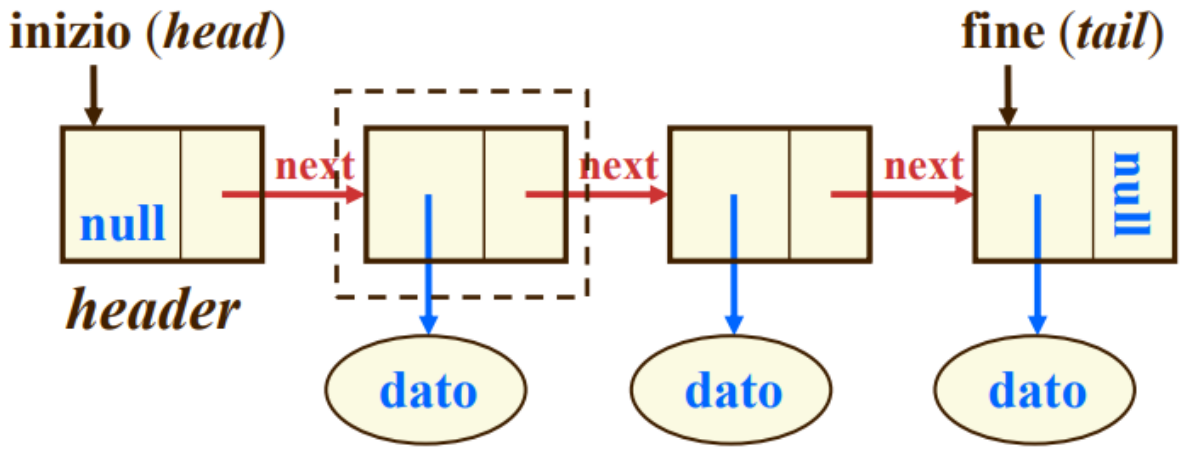
	In array ordinato	In array non ordinato
ricerca	O(log n)	O(n)
inserimento	O(n)	O(1)
rimozione	O(n)	O(n)

Scelta della Realizzazione

- **Array Non Ordinato**:
  - Frequenti inserimenti, ricerche e rimozioni sporadiche.
- **Array Ordinato**:
  - Dizionari statici utilizzati solo per ricerche.

LINKED LIST

Liste Concatenate



Definizione

- Una **lista concatenata** è una struttura dati che organizza elementi in **nodi** collegati e ordinati.
  - Ogni nodo contiene:
    - Un riferimento a un **elemento** (dato).
    - Un riferimento al **nodo successivo** (next).
  - Si memorizzano i riferimenti al **primo nodo** (**head**) e all'**ultimo nodo** (**tail**).

Caratteristiche Principali

- Il campo **next** dell'ultimo nodo è **null**.
- Si può utilizzare un nodo speciale chiamato **header** per semplificare la gestione dei casi limite: **lista vuota**.
  - head e tail puntano entrambi a header
- **scorrere la lista:** parto dall'header e si seguono i riferimenti a next
  - termina quando next è **null**

- **non al contrario**

ListNode

Non vengono **mai restituiti né ricevuti riferimenti ai nodi** (ListNode)**,** ma sempre ai dati contenuti nei nodi

- **information hiding:** ListNode **non visibile fuori** della classe LinkedList
- ListNode **classe interna privata**

Vantaggi

- Non esiste lista piena => solo ecc OutOfMemoryError se JVM occupa tutto lo spazio
  - non c'è bisogno di ridimensionare

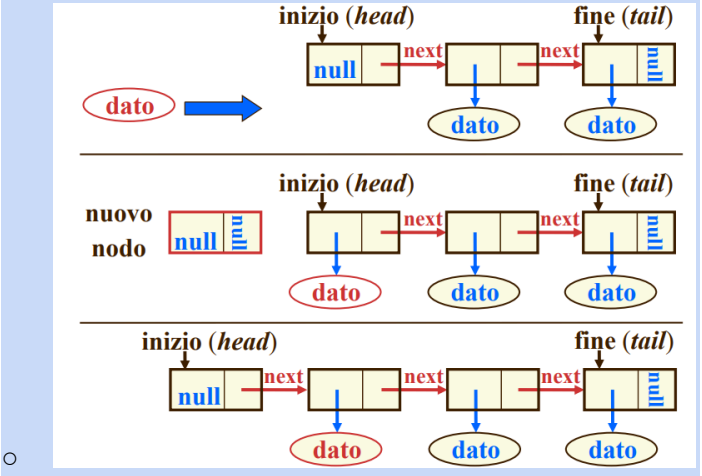
Svantaggi

- nodo occupa più spazio rispetto cella di array

Operazioni Fondamentali

Inserimento

- **addFirst**: Inserisce un elemento **all'inizio** della lista.
  - Complessità: O(1).



- **addLast**: Inserisce un elemento **alla fine** della lista.
  - Complessità: O(1) (con riferimento a tail).

Rimozione

- **removeFirst**: Rimuove il primo elemento della lista.
  - Complessità: O(1).
- **removeLast**: Rimuove l'ultimo elemento della lista.
  - Complessità: O(n) (richiede la ricerca del penultimo nodo).

Altri Metodi Utili

- **isEmpty**: Verifica se la lista è vuota.
- **makeEmpty**: Svuota la lista.
- **getFirst**: Restituisce il primo elemento.
- **getLast**: Restituisce l'ultimo elemento.

Iteratori per Liste Concatenate

Introduzione

- Un **iteratore** permette di **scorrere gli elementi** di una lista **senza accedere** direttamente ai nodi.
- L'iteratore si posiziona **tra due nodi,** partendo dopo l'header.

- Ha la propria **ADT**

Caratteristiche

- **classe interna privata**

Metodi Principali

- **next()**: Restituisce il dato del nodo successivo e sposta l'iteratore.
- **hasNext()**: Verifica se ci sono nodi successivi.
- **add(Object x)**: Inserisce un nuovo nodo prima della posizione corrente.
- **remove()**: Rimuove il nodo corrente e aggiorna la posizione.

Liste Doppiamente Concatenate

Definizione

- Ogni nodo contiene:
  - Un riferimento al nodo **precedente** (**prev**).
  - Un riferimento al nodo **successivo** (**next**).
- Vantaggi:
  - Simmetria strutturale.
  - Operazioni come **removeLast** diventano **O(1).**

Svantaggi

- Aumento dello **spazio** necessario per memorizzare i riferimenti.

Realizzazione di Altre Strutture con Liste Concatenate

Pila

- Implementata utilizzando l'inizio della lista concatenata.
  - Operazioni principali:
    - **push**: Aggiunge un elemento in testa **O(1)** con **addFirst**().
    - **pop**: Rimuove l'elemento in testa **O(1)** con **removeFirst()**

Coda

- Implementata utilizzando entrambe le estremità.
  - Operazioni principali:
    - **enqueue**: Inserisce alla fine **O(1)** con **addLast**()
    - **dequeue**: Rimuove all'inizio **O(1)** con **removeLast()**.

Hash Table con Liste Concatenate

Gestione delle Collisioni

- Utilizzo di **liste concatenate** per gestire **chiavi con lo stesso valore hash** (tecnica dei **bucket**).
- Prestazioni:
  - Caso ideale: **O(1)** con **distribuzione uniforme delle chiavi.**
  - Caso pessimo: **O(n)** se tutte le chiavi si mappano nello **stesso bucket.**

Implementazione

- Creare una tabella di **dimensione fissata**
  - se chiave è fuori dal range => **funzione di hash**

Vantaggi e Svantaggi delle Liste Concatenate

Vantaggi

- Non esistono problemi di dimensione fissa come negli array.
- Adattabilità a strutture dinamiche come pile e code.

Svantaggi

- Maggiore utilizzo di memoria rispetto agli array.
- Operazioni come l'accesso casuale sono inefficaci (O(n)).

⚠️ ACHTUNG!!!

Cose stupide ma stronze - STAI ATTENTA

Stringhe

Cosa stampa la riga di codice:

```
String river = “Mississipi”;
river.replace(“issip”,”our”);
System.out.println(river);
```

risposta: Mississipi

spiegazione: il metodo restituisce la stringa modificata. river, che invoca in metodo rimane INVARIATA

Cosa stampa la riga di codice se nella console scrivo 46 /n pluto:

```
int n = console.nextInt();
String name = console.nextLine();
System.out.println(name);
```

risposta: 46

spiegazione: in realtà l’utente non fa nemmeno a tempo a scrivere pluto perché nextInt legge 46 e vede /n => restituisce 46 e lascia nel flusso /n
nextLine() legge nel buffer /n => si ferma e restituisce la stringa letta fino a /n ossia una stringa vuota
soluzione: uso nextLine anche per il numero e converto

Copia di oggetti

Quali sono le dimensioni di rect1 dopo l'invocazione di setSize su rect2?:

```
Rectangle rect1 = new Rectangle(0, 0, 10, 20);
Rectangle rect2 = rect1;

rect2.setSize(30, 40);
```

risposta: 30, 40

Spiegazione: La copia è errata perché è stato copiato solo il riferimento, non l'oggetto. Di conseguenza, modifiche fatte su un esemplare si riflettono anche sull'altro, dato che entrambi ***puntano allo stesso oggetto*** *in memoria.*

Cicli

Funziona?:

```
do {
    System.out.println(...);
     double rate = ...; }
while (rate <= 0);
```

risposta: no

Spiegazione: al termine del ciclo, rate non è visibile. rate non è visibile nemmeno nella condizione. Soluzione => dichiarare rate prima del do

---

cosa restituisce il codice

```
int b = 1;
```

```
for(int i = 0; i < 5; i++){
   b  = b*i;
   break;
   System.out.println(b);
}
```

risposta: ERRORE segnalato dal compilatore: unreachable statement

*Spiegazione: il break non è condizionato (non dentro un blocco di decisione) quindi l'istruzione di stampa non verrà mai **eseguita***

Array

che valore viene stampato?:

```
BankAccount[] accounts = new BankAccount[10];
 System.out.println("Saldo nel conto 0"+ accounts[0].getBalance());
```

risposta: Errore NullPointerException

Spiegazione: Alla creazione dell'array viene creato l'array con tutti elementi con riferimento a null

---

cosa restituisce il codice

```
double[] values = {1, 2.3, 4.5, 5.6};
int indexToRemove = 1;

for (int i = values.length-1; i > indexToRemove ; i- - ) {
   values[i-1] = values[i];
}

values = resize(values, values.length - 1);
```

risposta: ERRORE logico

Spiegazione: *il codice doveva eliminare un elemento dall'array e spostare gli elementi successivi di una posizione indietro per mantenere l'ordine MA sposta l'ultimo elemento nella posizione prima, e poi di nuovo (ma il valore copiato è sempre lo stesso ossia l'ultimo)*

Classi

funziona?:

```
public class BankAccount{
 ...
    private int accountNumber;
    private int lastAssignedNumber;
 ...
    public BankAccount(){
        lastAssignedNumber++;
         accountNumber = lastAssignedNumber;
         balance = 0;
     }
}
```

risposta: no

Spiegazione: tutti i Bankaccount hanno una variabile di esemplare **lastAssignedNumber,** ogni volta che creo un BankAccount viene creata la copia di **lastAssignedNumber** che parte sempre da 1. Dovrei rendere **lastAssignedNumber statica**

Passaggio di parametri

cosa stampa?:

```
public class IncrementaNumero{
    public static void main(String[] args){
       int x = 10;
       increment1(x);
       System.out.println(x);
    }

    public static void increment1(int index) {
    index = index + 1;
     }
}
```

risposta: 10

Spiegazione: x viene copiato in increment1, quindi tutte le modifiche avvengono nel valore copiato, non su quello originale, che rimane invariato

---

<u>cosa succede ai riferimenti?:</u>

```
BankAccount a = new BankAccount(10);
BankAccount b = new BankAccount();
swapAccounts(a, b);

public static void swapAccounts(BankAccount x, BankAccount y) {
    BankAccount temp = x;
     x = y;
     y = temp;
}
```

risposta: a e b rimangono invariati

Spiegazione: x viene copiato il riferimento. I riferimenti originali rimangono fuori dal metodo. Avvengono modifiche all'oggetto al quale punta il riferimento, non il riferimento stesso

---

Polimorfismo

cosa succede?:

```
BankAccount account = new BankAccount();
((SavingAccount)account).addInterest();
```

risposta: ClassCastException
Spiegazione: account non è un tipo SavingAccount quindi non può essere convertito il SavingAccount e quindi non può invocare addInterest
soluzione: upcasting BankAccount account = new SavingAccount(); e aggiunta del controllo con **instanceof** per maggiore sicurezza

---

Questo metodo è l'overring di equals della classe Object?:

```
public boolean equals(BankAccount other){
    return balance == other.balance;
}
```

risposta: no, è un sovraccarico
Spiegazione: la firma non è la stessa, per sovrascrivere dovrei passare come parametro un Object e successivamente fare un casting a Bakaccount

---

PILA

è il metodo corretto per creare una pila sicura contenente solo Stringhe?:

```
public class StringArrayStack implements Stack{
    public void push(String obj){
        if (vSize == v.length)
            v = resize(v, 2*v.length);
        v[vSize++] = obj;
    }
}
```

risposta: no

Spiegazione: Non vengono implementati tutti i metodi dell'interfaccia Stack perché la firma di push ha come parametro un Object → non una Stringa . Metodo corretto sfrutta l'ereditarietà e controlla se l'oggetto inserito è un'istanza di String

---

Linked List

è il modo corretto per accedere all'head di una LinkedList?:

```
public class LinkedList {
    public ListNode getHead(){
        return head;
    }
}
```

risposta: no
Spiegazione: Si viola l'incapsulamento se ListNode è public, se è privata i nodi sono inaccessibili e non riesco

---

🤝 Altre Questionnnn

Autovalutazione 1

Domanda 1

Qual è il valore del più piccolo numero negativo rappresentabile come int in Java?

a. -2^{31}
b. -2^{32}+1
c. 2^{31}-1
d. Nessuna delle altre risposte è corretta

a. -2^31

Domanda 2

Cosa succede quando si divide un numero intero in Java per 0?

a. Si ottiene il valore NaN (Not a Number)
b. Si genera un'eccezione ArithmeticException
c. Il risultato è 0
d. Nessuna delle altre risposte è corretta

b. si genera un'eccezione ArithmeticException

Domanda 3

Quale di questi tipi di dati Java occupa più memoria?

a. int
b. short
c. double
d. float

c. double

Domanda 4

In Java, quale sarà l'output del seguente codice?

```
int x = 5;
int y = 10;
System.out.println(x + y + " è il risultato");
```

a. 15 è il risultato
b. 510 è il risultato
c. 5 + 10 è il risultato
d. Nessuna delle altre risposte è corretta

a. 15 è il risultato

Domanda 5

In un sistema binario, qual è il valore decimale del numero 10101010?
a. 128
b. 170
c. 256
d. 85

Domanda 6

In Java, la parola chiave this:
a. Si riferisce alla classe corrente
b. Si riferisce all'istanza corrente della classe
c. Può essere usata solo nei metodi statici
d. Nessuna delle altre risposte è corretta

Domanda 7

Quale di questi non è un operatore logico in Java?

a. &&
b. ||
c. !
d. ??

Domanda 8

Quale sarà il risultato del seguente codice?

```
int a = 10;
int b = 20;
System.out.println(++a + b++);
```

a. 31
b. 30
c. 32
d. Nessuna delle altre risposte è corretta

Domanda 9

Nel modello di Von Neumann, il bus della CPU:

a. Connette solo l'unità di controllo e l'ALU
b. Trasporta dati e indirizzi tra i vari blocchi del sistema
c. È responsabile del calcolo delle istruzioni
d. Nessuna delle altre risposte è corretta

Domanda 10

In un sistema a complemento a 2 con 8 bit, quale valore rappresenta il numero -128?

a. 10000000
b. 01111111
c. 11111111
d. Il numero non può essere rappresentato in 8 bit

Domanda 1

In Java, il valore del più grande numero rappresentabile come float è approssimativamente:

a. 3.4028235×10383.4028235 \times 10^{38}
b. 1.7976931348623157×103081.7976931348623157 \times 10^{308}
c. 231-12^{31} - 1
d. Nessuna delle altre risposte è corretta

Domanda 2

In Java, quale sarà l'output del seguente codice?

```
int a = 0b1010; // valore binario

int b = 0xF;    // valore esadecimale

System.out.println(a + b);
```

a. 25
b. 15
c. 20
d. Nessuna delle altre risposte è corretta

Domanda 3

In un sistema a complemento a 2 con 8 bit, qual è la rappresentazione binaria del numero -45?

a. 1101 0011
b. 0010 1101
c. 1101 0010
d. Nessuna delle altre risposte è corretta

Domanda 4

Dato il seguente frammento di codice Java:

```
byte b = (byte) 130;

System.out.println(b);
```

Quale sarà l'output?

a. 130
b. -126
c. 0
d. Nessuna delle altre risposte è corretta

(*In Java, il tipo* `byte` *ha un intervallo da -128 a 127. Il valore 130 va fuori intervallo e subisce un overflow ciclico: 130 - 256 = -126.*)

Domanda 5

In un sistema IEEE754 a singola precisione, il numero più vicino a 1.0 che può essere rappresentato è:

a. 1.0 + 2^{-23}
b 1.0 + 2^{-24}
c. 1.0 + 2^{-52}
d. Nessuna delle altre risposte è corretta

Domanda 6

Qual è il risultato della seguente operazione in Java?

```
double x = 0.1 + 0.2;

System.out.println(x == 0.3);
```

a. true
b. false
c. Genera un errore di compilazione
d. Nessuna delle altre risposte è corretta

Domanda 7

Nel formato binario a complemento a 2 con 16 bit, quale rappresentazione corrisponde al valore decimale -32768?

a. 1000 0000 0000 0000
b. 0111 1111 1111 1111
c. 1111 1111 1111 1111
d. Nessuna delle altre risposte è corretta

Domanda 8

In Java, se una variabile x è di tipo float e contiene il valore 1.0f quale sarà il risultato dell'espressione:

```
System.out.println(x / 0);
```

a. Infinity
b. NaN
c. Genera un'eccezione di tipo ArithmeticException
d. Nessuna delle altre risposte è corretta

Domanda 9

Dato il seguente numero binario a complemento a 2 a 10 bit: 1111 1011 01 qual è la rappresentazione in base 10?

a. -77
b. 493
c. -493
d. Nessuna delle altre risposte è corretta

Domanda 10

Quale delle seguenti affermazioni sul modello di Von Neumann è corretta?

a. Il modello prevede memorie separate per dati e istruzioni
b. L'unità di controllo e l'ALU fanno parte dello stesso blocco logico collegato al bus
c. Nel modello di Von Neumann non esiste la memoria cache
d. Nessuna delle altre risposte è corretta

autovalutazione 2

Domanda 1

Che cosa restituisce il seguente metodo quando il parametro r è un valore negativo?

```
private static int calcola(int r, int s) {
  int q = 1;
  int i = 0;
  while (i < r) {
    q *= s;
    i++;
  }
```

```
    return q;
}
```

Scegli un'alternativa:

a. restituisce 1

b. genera un ciclo infinito

c. restituisce il numero r^s

d. restituisce il numero s^r

e. genera un errore in compilazione

restituisce 1

Domanda 2

Quale sarà l'output del seguente codice?

```
int a = 5;
int b = -2;
int c = a % b;
System.out.println(c);
```

Scegli un'alternativa:

a. 1

b. -1

c. 2

d. -2

e. genera un errore in compilazione

Domanda 2.bis

Quale sarà l'output del seguente codice?

```
int a = -5;
int b = 2;
```

```
int c = a % b;
System.out.println(c);
```

Scegli un'alternativa:

a. 1

b. -1

c. 2

d. -2

e. genera un errore in compilazione

(*anche con -5 e -2*)

Domanda 3

Che cosa restituisce il seguente metodo quando è invocato con a=6 e b=0?

```
public static boolean verifica(int a, int b) {
   return (a % b == 0) && (b != 0);
}
```

Scegli un'alternativa:

a. true

b. false

c. genera un errore in compilazione

d. lancia un'eccezione ArithmeticException

Domanda 4

Che cosa stampa il seguente frammento di codice?

```
int a = 10;
int b = 5;
System.out.println((a > b) && (b > 0) || (a < 0));
```

Scegli un'alternativa:

a. true

b. false

c. lancia un'eccezione

d. nessuna delle altre risposte è corretta

Domanda 5

Che cosa succede se confrontiamo due stringhe in Java con l'operatore `==` invece del metodo `.equals()`?

a. Verifica l'uguaglianza del contenuto delle stringhe

b. Verifica se i riferimenti puntano allo stesso oggetto

c. Genera un errore di compilazione

d. Nessuna delle altre risposte è corretta

Domanda 6

Quale sarà il risultato della seguente espressione booleana, sapendo che a = false e b = true?

!a || (a && b) && !b

Scegli un'alternativa:

a. true

b. false

c. genera un errore in compilazione

d. nessuna delle altre risposte è corretta

Domanda 7

In un confronto lessicografico tra due stringhe in Java, quale delle seguenti affermazioni è vera?

a. Se la prima stringa è più lunga, il confronto restituisce sempre true

b. Si confrontano i caratteri delle due stringhe finché non si trova una differenza o si esauriscono i caratteri della stringa più corta
c. Le stringhe vengono convertite in numeri interi prima di essere confrontate
d. Nessuna delle altre risposte è corretta

Domanda 8

Che cosa stampa il seguente codice?

```
String str1 = "ciao";
String str2 = str1 + " mondo";
if (str2.equals("ciao mondo")) {
   System.out.println(1);
} else {
   System.out.println(2);
}
```

Scegli un'alternativa:
a. 1
b. 2
c. genera un errore in compilazione
d. nessuna delle altre risposte è corretta

Domanda 9

Che cosa accade se una variabile d'istanza in Java non viene inizializzata esplicitamente?
a. Genera un errore in compilazione
b. Genera un'eccezione in fase di esecuzione
c. Viene inizializzata automaticamente con un valore predefinito
d. Nessuna delle altre risposte è corretta

Domanda 10

Quale sarà il valore della variabile res dopo l'esecuzione del seguente codice?

```
int x = 5;
int y = 3;
boolean res = (x + y) > 7 && (x * y) < 20;
```

Scegli un'alternativa:

a. true
b. false
c. genera un errore in compilazione
d. nessuna delle altre risposte è corretta

Autovalutazione 3

Domanda 1

Dato il metodo seguente, quale affermazione è corretta?

```
private static int sommaPari(int[] b) {
  int somma = 0;
  for (int i = 0; i < b.length; i++) {
    if (b[i] % 2 == 0) {
      somma += b[i];
    }
  }
```

```
    return somma;
}
```

Scegli un'alternativa:

a. Il metodo restituisce il prodotto di tutti gli elementi pari dell'array.
b. Il metodo restituisce la somma di tutti gli elementi dispari dell'array.
c. Il metodo restituisce la somma di tutti gli elementi pari dell'array.
d. Il metodo restituisce l'elemento minimo dell'array.
e. Nessuna delle altre risposte è corretta.

Domanda 2

Quale risultato produce il seguente frammento di codice?

```
String s = "abc def ghi";

String[] parole = s.split(" ");

System.out.println(parole[3]);
```

Scegli un'alternativa:

a. Stampa "ghi".
b. Lancia l'eccezione ArrayIndexOutOfBoundsException.
c. Stampa "def ghi".
d. Lancia l'eccezione NullPointerException.
e. Nessuna delle altre risposte è corretta.

Domanda 3

Che tipo di errore provoca l'esecuzione della classe seguente?

```
public class Example {
```

```
    public static void main(String[] args) {
        int[] v = new int[5];
        for (int i = 0; i <= 5; i++) {
            v[i] = i * 2;
        }
    }
}
```

Scegli un'alternativa:

a. NullPointerException

b. ArrayIndexOutOfBoundsException

c. Nessun errore: il programma funziona correttamente.

d. ArithmeticException

e. Nessuna delle altre risposte è corretta.

Domanda 4

Quale valore restituisce il seguente metodo quando invocato con il parametro n = 3?

```
private static int prodotto(int n) {
    int risultato = 1;
    for (int i = 1; i <= n; i++) {
        risultato *= i;
    }
    return risultato;
}
```

Scegli un'alternativa:

a. 6
b. 3
c. 0
d. 9
e. Nessuna delle altre risposte è corretta.

Domanda 5

Cosa succede eseguendo il seguente frammento di codice?

```
String s = null;

if (s.equals("test")) {

   System.out.println("Esempio corretto");

} else {

   System.out.println("Esempio errato");

}
```

Scegli un'alternativa:

a. Stampa "Esempio errato".
b. Lancia l'eccezione NullPointerException.
c. Stampa "Esempio corretto".
d. Lancia l'eccezione StringIndexOutOfBoundsException.
e. Nessuna delle altre risposte è corretta.

Autovalutazione 4

Domanda 1

Con riferimento alla complessità dell'algoritmo di ricerca binaria su un array ordinato di n elementi, quale affermazione è corretta?

a. È O(n)nel caso peggiore
b. È O(1) nel caso migliore
c. È O(logn) nel caso peggiore
d. Nessuna delle affermazioni è corretta

Domanda 2

Quale è la complessità temporale dell'algoritmo di ordinamento per selezione nel caso migliore?

a. O(nlogn)
b. O(n^2)
c. O(n)
d. Dipende dal numero di elementi già ordinati

Domanda 3

Dato il seguente codice, quale sarà l'output?

```
public class Test {
   public static int ricorsione(int n) {
      if (n == 0)
         return 1;
      return n * ricorsione(n - 1);
   }

   public static void main(String[] args) {
      int risultato = ricorsione(4);
      System.out.println(risultato);
   }
}
```

a. 24
b. 4
c. 0
d. 1

Domanda 4

Qual è la complessità temporale asintotica del mergesort, dato n, nel caso medio?

a. O(n2)
b. O(logn)
c. O(nlogn)
d. Nessuna delle precedenti

Domanda 5

Dati due algoritmi i cui tempi di esecuzione sono dati dalle seguenti funzioni, quale ha una complessità asintotica migliore?

1. f1(n)=3n+logn
2. f2(n)=n^2+n

a. f1(n)
b. f2(n)
c. Sono equivalenti
d. Dipende dal valore di nn

Domanda 6

Quale sarà l'output del seguente codice?

```
public class Test {
   public static void main(String[] args) {
      int[] array = {1, 2, 3, 4};
```

```
        for (int i = 0; i <= array.length; i++) {
            System.out.print(array[i] + " ");
        }
    }
}
```

a. 1 2 3 4
b. 1 2 3 4 seguito da un'eccezione
c. Genera un errore di compilazione
d. Nessuna delle precedenti

Domanda 7

Quale delle seguenti è una proprietà necessaria per un algoritmo ricorsivo?

a. Deve avere almeno un caso base
b. Deve terminare dopo un numero finito di passi
c. Deve chiamare se stesso con un problema più semplice
d. Tutte le precedenti

Domanda 8

Se un algoritmo ha una complessità O(1), cosa significa?

a. Il tempo di esecuzione non dipende dalla dimensione dell'input
b. Il tempo di esecuzione è lineare rispetto alla dimensione dell'input
c. È il caso migliore di un algoritmo qualsiasi
d. È più lento di un algoritmo O(logn)

Domanda 9

Qual è il risultato della seguente funzione ricorsiva per n=3e m=2?

```
public static int somma(int n, int m) {
```

```
  if (m == 0)
    return n;
  return somma(n + 1, m - 1);
}
```

a. 6
b. 5
c. 3
d. Lancia un'eccezione

Autovalutazione 5

Domanda 1

Siano A e B due pile. Cosa realizza il seguente codice?

```
while (!A.isEmpty()) {
  B.push(A.pop());
}
while (!B.isEmpty()) {
  System.out.println(B.pop());
}
```

a. Copia il contenuto di A in B rovesciandone l'ordine.
b. Copia il contenuto di A in B mantenendone l'ordine.
c. Lancia un'eccezione se B è piena.
d. Stampa il contenuto di A in ordine inverso.

Domanda 2

In Java, una sottoclasse:

a. Può accedere solo ai metodi public e protected della superclasse.
b. Può accedere ai metodi private della superclasse.
c. Non eredita i metodi protected della superclasse.
d. Non può ridefinire i metodi ereditati.

Domanda 3

Siano A e B due code. Quale effetto produce il seguente codice?

```
while (!A.isEmpty()) {
  B.enqueue(A.dequeue());
}
```

a. Copia il contenuto di A in B mantenendo l'ordine.
b. Rovescia il contenuto della coda A in B.
c. Lancia un'eccezione di tipo QueueFullException se B è piena.
d. Nessuna delle altre risposte è corretta.

Domanda 4

Un'interfaccia in Java:

a. Può contenere metodi private.
b. È un tipo di classe astratta con implementazioni parziali.
c. Contiene solo la dichiarazione di metodi pubblici e costanti statiche finali.
d. È la documentazione pubblica di una classe.

Domanda 5

Il seguente frammento di codice:

```
A.push(10);
A.push(20);
A.push(30);
A.pop();
System.out.println(A.top());
```

a. Lancia un'eccezione perché la pila è vuota.
b. Stampa 30.
c. Stampa 20.
d. Stampa 10.

Domanda 6

Che complessità ha l'operazione push in una pila realizzata con un array ridimensionabile?

a. È sempre O(1).
b. È O(n) nel caso peggiore, ma O(1) nella media.
c. È sempre O(log n).
d. Nessuna delle altre risposte è corretta.

Domanda 7

Il metodo equals della classe Object:

a. Confronta due oggetti verificando se hanno lo stesso riferimento in memoria.
b. Deve essere sovrascritto per confrontare il contenuto di due oggetti.
c. Non può essere sovrascritto.
d. a e b sono corrette.

Domanda 8

Il seguente codice:

```
import java.io.File;
import java.io.FileReader;

FileReader reader = new FileReader("input.txt");
reader.close();
```

a. Compila correttamente, ma lancia un'eccezione in esecuzione.
b. Compila ed esegue correttamente se il file input.txt esiste.
c. Non compila perché il metodo close non è disponibile in FileReader.
d. Nessuna delle altre risposte è corretta.

(eccezione in compilazioe)

Domanda 9

Quale delle seguenti affermazioni è **vera**?

a. Una classe può estendere più classi in Java.
b. Una classe può implementare più interfacce in Java.
c. Una classe può estendere una classe e implementare un'interfaccia contemporaneamente.
d. b e c sono corrette.

Domanda 11

In Java, qual è la complessità temporale della ricerca binaria in un array ordinato di n elementi?

a. O(1)
b. O(log n)
c. O(n)
d. Nessuna delle altre risposte è corretta.

Domanda 12

Il seguente codice:

```
Object obj = 10;
System.out.println((Integer) obj + 5);
```

a. Lancia un'eccezione ClassCastException.
b. Stampa 15.
c. Non compila perché Object non è un tipo numerico.
d. Nessuna delle altre risposte è corretta.

Domanda 13

Per sovrascrivere un metodo ereditato in una sottoclasse:

a. Il metodo deve avere la stessa firma e non deve essere final.
b. Il metodo deve avere un'intestazione diversa rispetto a quella nella superclasse.
c. Il metodo deve essere dichiarato come static.
d. Il metodo deve avere una visibilità maggiore rispetto alla versione nella superclasse.